

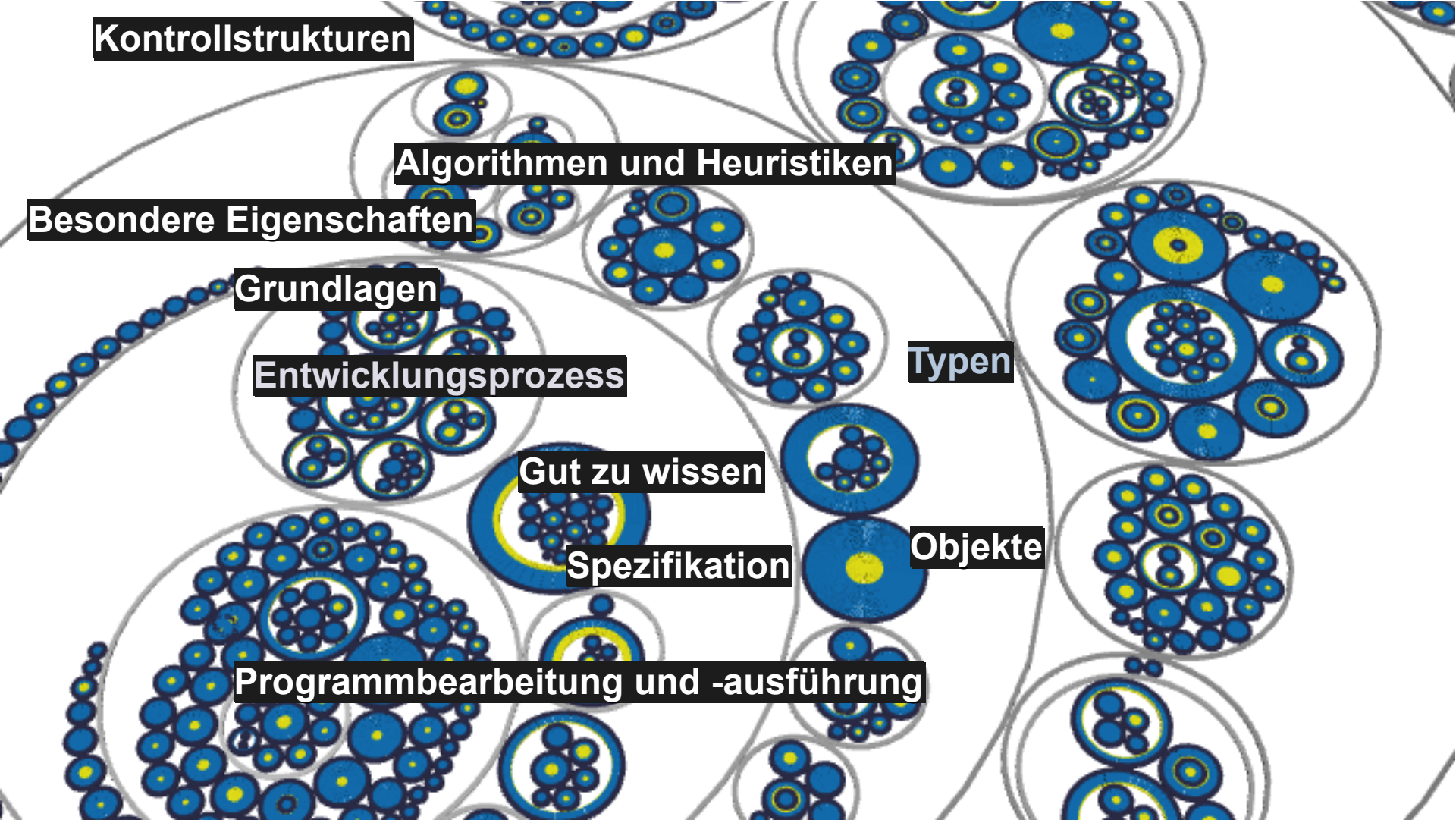
UNIVERSITÄT
LEIPZIG

Softwareentwicklung

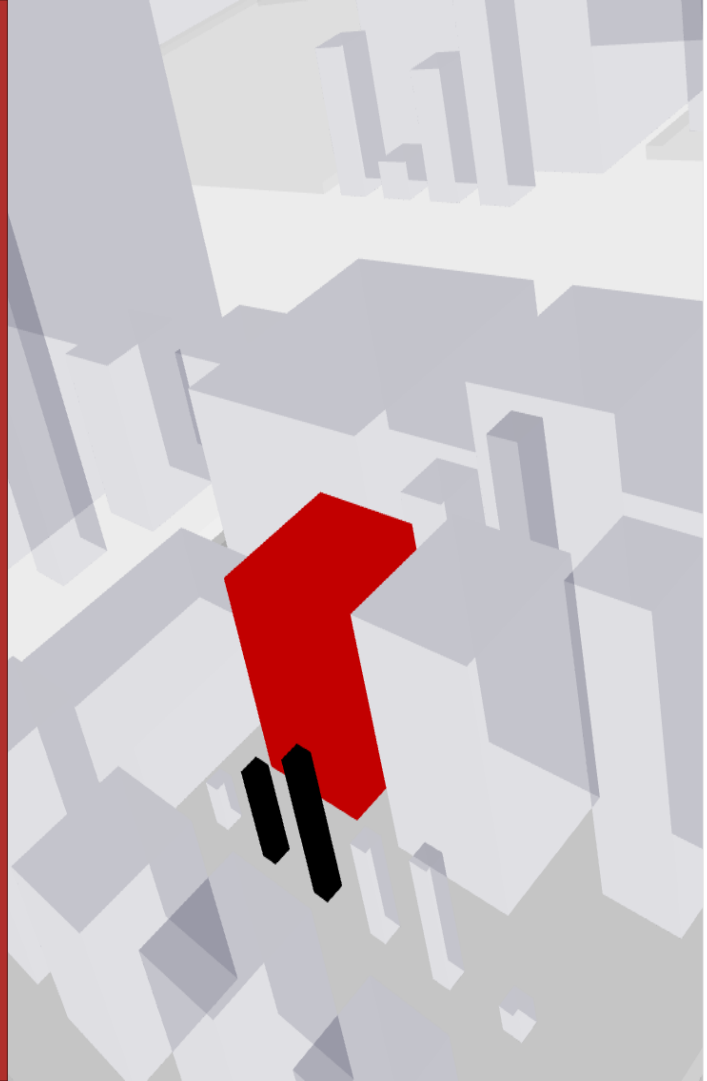
Einführung in die Informatik

Leipzig, 2026

Dr. Nico Willert Stark / Prof. Dr. Ulrich Eisenecker


Kontrollstrukturen
Algorithmen und Heuristiken
Besondere Eigenschaften
Grundlagen
Typen
Entwicklungsprozess
Gut zu wissen
Objekte
Spezifikation
Programmbearbeitung und -ausführung

BESONDERE EIGENSCHAFTEN VON SOFTWARE



CodeViewer

```
39  public class ScriptEditorProperty extends PropertyBean implements
40          PropertyControl, ActionListener {
41
42      public interface ScriptEditorStarter extends MindMapControllerPlugi
43          String startEditor(String scriptInput);
44      }
45
46      String description;
47
48      String label;
49
50      String script;
51
52      JButton mButton;
53      final JPopupMenu menu = new JPopupMenu();
54
55      private final MindMapController mMindMapController;
56
57      private static java.util.logging.Logger logger = null;
58
59      /**
60       */
61      public ScriptEditorProperty(String description, String label,
62              MindMapController pMindMapController) {
63          super();
64          this.description = description;
65          this.label = label;
66          mMindMapController = pMindMapController;
67          if (logger == null) {
68              logger = mMindMapController.getFrame().getLogger(
69                      this.getClass().getName());
70          }
71          mButton = new JButton();
72          mButton.addActionListener(this);
73          script = "";
74      }
75
76      public String getDescription() {
77          return description;
78      }
79
80      public String getLabel() {
81          return label;
82      }
```

SOFTWARE IST BESONDERS

Balzert nennt sieben Gründe, warum Software etwas Besonderes ist

(Balzert and Balzert, Software-Entwicklung.)

- Immaterielles Produkt
- Verschleißt nicht
- Altert
- Für Software gibt es keine Ersatzteile
- Nicht durch physikalische Gesetze Gesetze eingeschränkt
- Formbarkeit
- Messbakeit

SOFTWARE IST BESONDERS

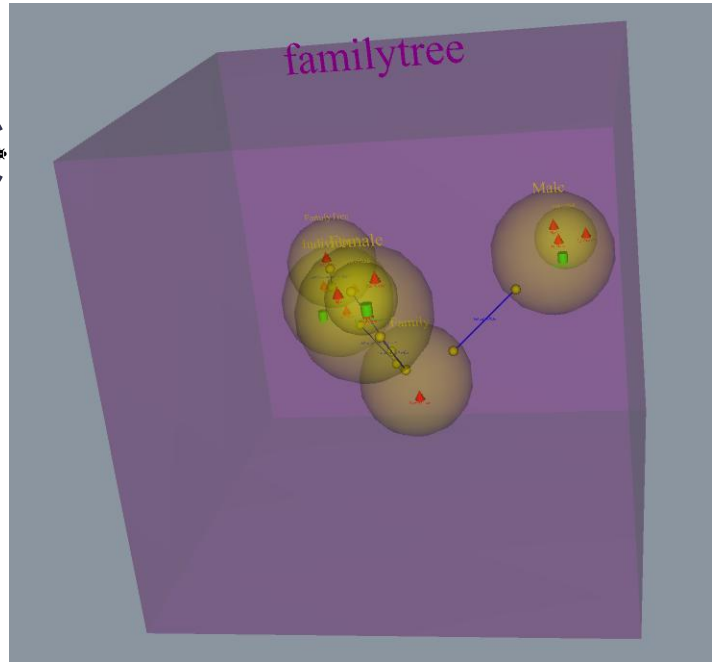
IMMATERIELLES PRODUKT

- Binärcode oder Ausführungsabläufe im Speicher, sind unsichtbar und können nicht direkt beeinflusst werden
- Code und Dokumentation weisen zumeist eine Diskrepanz auf, welche durch Wartung vergrößert wird.
- Es ist schwierig, den Entwicklungsfortschritt eines Softwareprojekts zuverlässig einzuschätzen, was für das Management und die Kunden ein großes Problem darstellt.

SOFTWARE IST BESONDERS

IMMATERIELLES PRODUKT

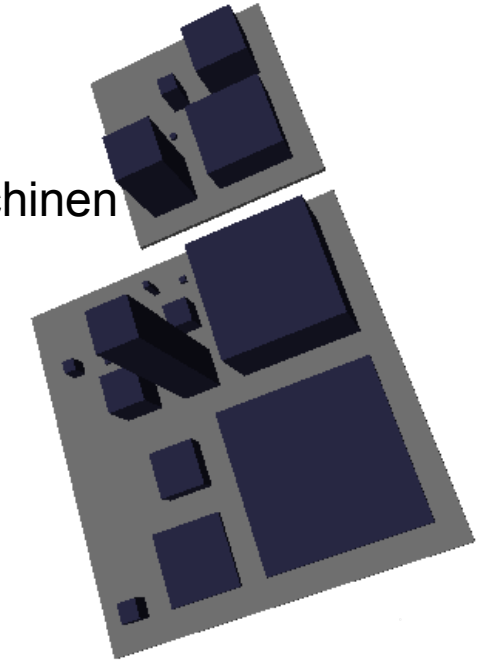
- Die Forschungsgruppe „Visual Software Analytics“ beschäftigt sich damit, Software und Artefakte der Softwareentwicklung sichtbar zu machen.
- Weitere Informationen finden Sie unter
  - http://home.uni-leipzig.de/svis/
  - Anwendungsbeispiele zum Selberausprobieren, einschlägige Veröffentlichungen, Informationen zu Industriepartnern, …

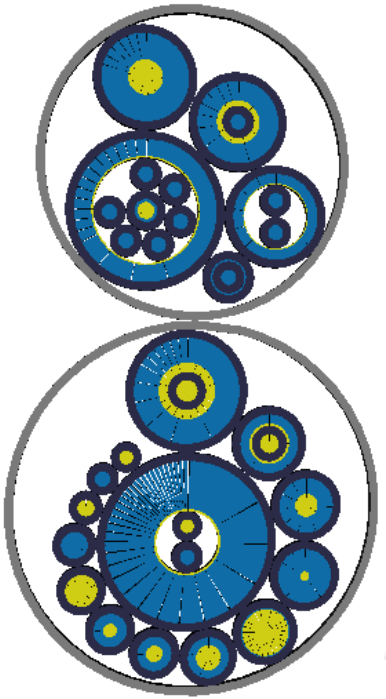


SOFTWARE IST BESONDERS

IMMATERIELLES PRODUKT

- https://github.com/cjlin1/libsvm
- Bibliothek für Support-Vektor-Maschinen
  - 2 packages
  - 34 classes
  - 143 attributes
  - 136 methods
  - 2,333 statements

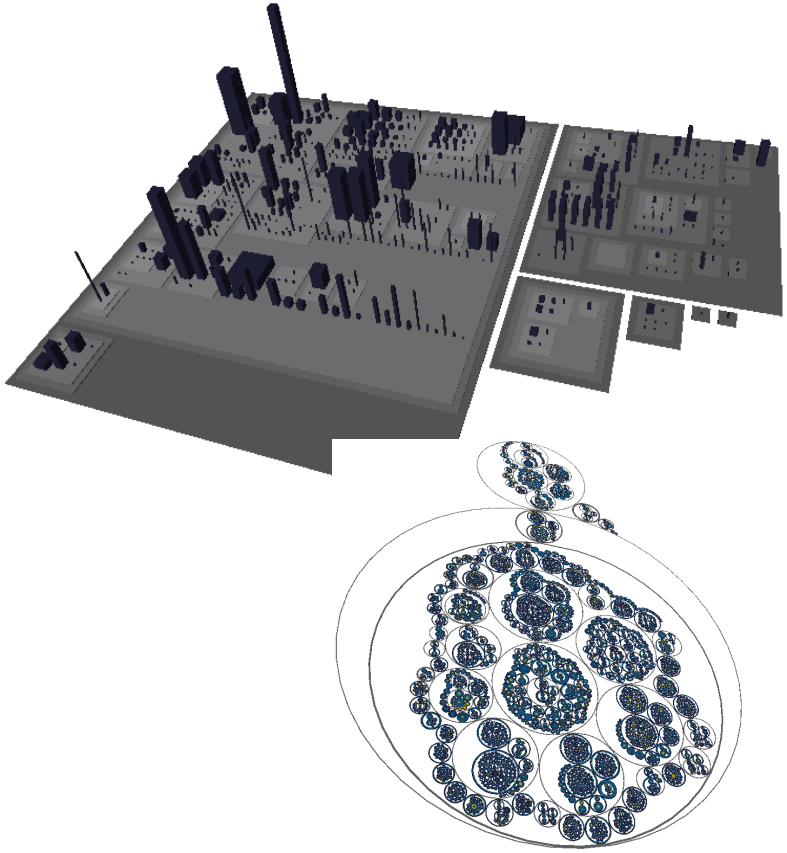




SOFTWARE IST BESONDERS

IMMATERIELLES PRODUKT

- https://github.com/roboguice/roboguice
- Google Guice auf Android 3.0
- Bibliothek für die Android-Entwicklung
  - 71 packages
  - 3,203 classes
  - 74,173 attributes
  - 2,129 methods
  - 35,295 statements



SOFTWARE IST BESONDERS

KEIN VERSCHLEIß

- Software kann beliebig oft ausgeführt werden, ohne dass sie dabei verschleißt.
- Für den Austausch verschlissener Teile ist keine Wartung erforderlich
- Die Wartung ist für die Fehlerbehebung, die Behebung von Fehlern und die Anpassung an geänderte Anforderungen erforderlich.

ALTERUNG

- Software nutzt sich nicht ab, aber ihre Umgebung und die Anforderungen können sich erheblich ändern
- Wenn die Software nicht ständig an die veränderten Rahmenbedingungen angepasst wird, veraltet sie und wird irgendwann überholt.

SOFTWARE IST BESONDERS

KEINE ERSATZTEILE

- Bei technischen Produkten werden verschlissene oder defekte Teile durch vorgefertigte Ersatzteile ersetzt.
- Bei Software ist das anders
  - Die Teile nutzen sich nicht ab und bekommen keine Mängel, auch wenn sie möglicherweise von Anfang an Fehler oder Mängel aufweisen.
  - Komponenten werden nicht mehrfach erstellt und anschließend ausgetauscht, sondern:
    - Komponenten können entwickelt bzw. geändert werden, um Fehler zu beheben oder sicherzustellen, dass sie nicht erneut auftreten.
    - Diese müssen hinsichtlich ihrer Schnittstelle und Funktionalität gleichwertig sein.
    - Empirische Erkenntnisse deuten jedoch darauf hin, dass unabhängige Entwicklungsteams häufig ähnliche Fehler produzieren.

SOFTWARE IST BESONDERS

KEINE EINSCHRÄNKUNGEN DURCH PHYSIKALISCHE GESETZE

- Software ist ein künstliches Produkt des menschlichen Geistes.
  - Es unterliegt also nicht den Gesetzen der Physik
- Die Software bietet eine Vielzahl von Gestaltungsmöglichkeiten
  - z. B. Computerspiele
- Für alle Interessierten
  - Popper, Karl R. Auf der Suche nach einer besseren Welt: Vorträge und Aufsätze aus dreissig Jahren. München: Piper, 1984. (worlds I, II, and III)
  - Deutsch, David. The Fabric of Reality. London; New York: Penguin Books, 1998. (bis einschließlich Zeitreisen mithilfe virtueller Realität)

SOFTWARE IST BESONDERS

FORMBARKEIT

- Software lässt sich in der Regel einfacher und schneller ändern als ein technisches Produkt
- Änderungen an technischen Produkten erfordern oft neue Werkzeuge – selbst wenn die Architektur des Produkts klar beschrieben und bekannt ist
- Wenn die Software gut strukturiert und modular aufgebaut ist, lassen sich Änderungen relativ einfach und schnell umsetzen.
  - Allerdings: Ein Softwarewechsel ist mit Kosten verbunden, die von verschiedenen Faktoren abhängen, z. B. von der Komplexität, dem Grad der Modularisierung und der Verfügbarkeit von Entwicklern.

SOFTWARE IST BESONDERS

MESSBAKEIT

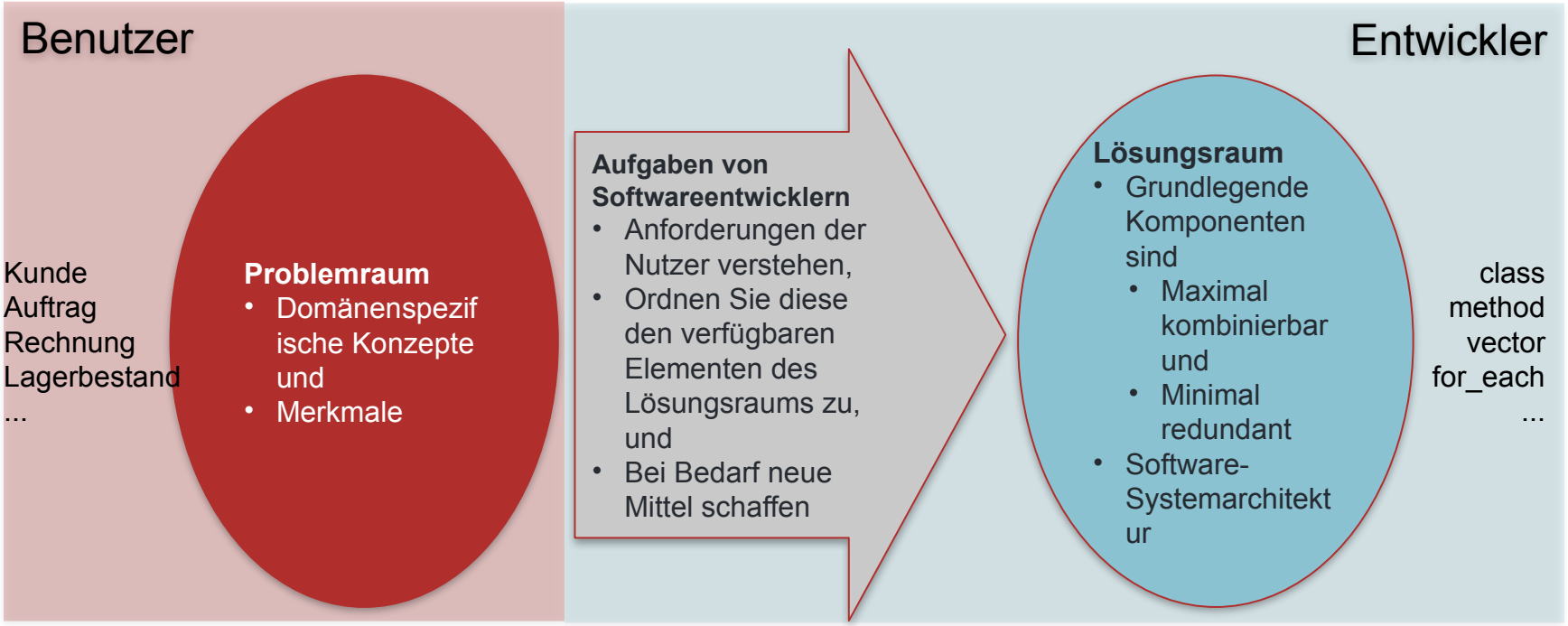
- In der Regel lassen sich technische Produkte präzise messen, z. B. Gewicht, Größe, Geschwindigkeit, und auf der Grundlage dieser Messwerte miteinander vergleichen.
- Bei Software ist das anders.
  - Softwarequalität lässt sich nur schwer definieren und quantifizieren
  - Ähnlich wie bei Begriffen wie Gesundheit oder Intelligenz
- Grundlegende Merkmale von Messungen:
  - Zuverlässigkeit – Ist eine Messung präzise und wiederholbar?
  - Validität – Misst eine Messung tatsächlich die Eigenschaft, die von Interesse ist?
  - Objektivität – Ist eine Messung unabhängig von der Person, die sie durchführt?

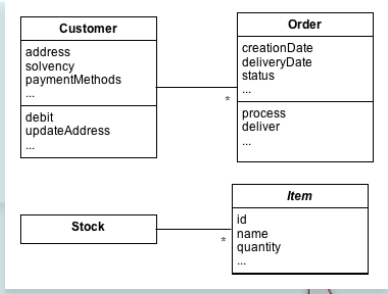
GRUNDLAGEN



By Tobias Deml - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=80409304

SEMANTISCHE LÜCKE IN DER SOFTWAREENTWICKLUNG





Die Notation erleichtert die Kommunikation unter Ingenieuren, nicht jedoch zwischen Kunden und Ingenieuren!

Benutzer

Kunde
Auftrag
Rechnung
Lagerbestand
...

Problemraum
- Domänenspezifische Konzepte und
- Merkmale

Aufgaben von Softwareentwicklern
- Anforderungen der Nutzer verstehen,
- Ordnen Sie diese den verfügbaren Elementen des Lösungsraums zu, und
- Bei Bedarf neue Mittel schaffen

Entwickler

Lösungsraum
- Grundlegende Komponenten sind
  - Maximal kombinierbar und
  - Minimal redundant
- Software-Systemarchitektur

class
method
vector
for_each
...

PROGRAMMIERPARADIGMEN

- Kognitive Prozesse von Softwareentwicklern lenken und begrenzen sowie deren Arbeitsergebnisse beeinflussen
- Softwareentwickler wenden bestimmte konzeptionelle Modelle (Programmierparadigmen), Sprachen und Notationen an, um funktionale und technische Modelle zu erstellen.
  - Imperatives und prozedurales Paradigma
  - Objektorientiertes Paradigma
  - Regelbasiertes Paradigma
  - …

SOFTWARE-LEBENSZYKLUS

- Planung: Anforderungen ermitteln und analysieren, Aufwand abschätzen
- Definition: Geschäftsanforderungen modellieren
- Entwurf: Technische Lösung des Modells
- Umsetzung: Softwareprodukt erstellen
- Test: Testen des Softwareprodukts in Teilen und als Ganzes
- Abnahme und Einweisung: Übergabe des Softwareprodukts, Inbetriebnahme und Schulung der Anwender
- Wartung und Pflege: Fehlerbehebung, Anpassung von Softwareprodukten an geänderte Anforderungen und Einarbeitung neuer Anforderungen
- Stilllegung: Das Softwareprodukt wird nicht mehr verwendet und wird möglicherweise durch ein anderes System ersetzt.

AKTIVITÄTEN UND ARTEFAKTE

- Planung: Machbarkeitsstudie (= Leistungsbeschreibung, Kostenvoranschlag, Projektplan), Software-Anforderungsspezifikation (SRS)
- Definition: Spezifikationen, technisches Produktmodell, Prototyp oder Pilotsystem
- Entwurf: Produktentwurf (= Softwarearchitektur, Spezifikation der einzelnen Systemkomponenten)
- Implementierung: Programmquellcode, ausführbares Programm, System- und Benutzerdokumentation
- Test: Testfälle und Testtreiber

ENTWICKLUNGSDAUER IM VERGLEICH ZUR NUTZUNGSDAUER

- Entwicklungsdauer wird immer kürzer
  - Vor einigen Jahrzehnten waren 1–2 Jahre (oder sogar noch länger) für ein Softwareentwicklungsprojekt durchaus üblich.
  - Heutzutage sind 3 Monate bis zu 1 Jahr der Standard („Internetzeit“)
- Nutzungsdauer ist in der Regel deutlich länger
  - In vielen Fällen bis zu 30 Jahre und mehr
  - Das schnellere Tempo der technischen und organisatorischen Veränderungen führt jedoch auch zu einer Verkürzung der Nutzungsdauer.

AUFWANDSVERTEILUNG

- Die Zahlen variieren
  - 20 % Entwicklung, 80 % Wartung und Pflege
  - 33 % Entwicklung, 67 % Wartung und Pflege
  - Die Ausgaben für Wartung und Pflege sind zwei- bis viermal so hoch wie die für die Entwicklung (nach Balzert und Balzert, „Software-Entwicklung“).
- Im Allgemeinen
  - Wartungs- und Instandhaltungskosten steigen mit dem Alter und dem Umfang eines Softwareprodukts

FEHLER UND DAMIT VERBUNDENE KOSTEN

- Jede Software ist mehr oder weniger von Fehlern betroffen
- Zu den Ursachen für Fehler gehören
  - Falsche oder fehlende Anforderungen und
  - Programmierfehler
- In der Regel
  - Je früher ein Fehler im Software Lebenszyklus erkannt und behoben wird, desto geringer sind die damit verbundenen Kosten.
  - Je später ein Fehler im Software Lebenszyklus entdeckt und behoben wird, desto höher sind die daraus resultierenden Kosten
  - Nur anhand Entwicklungszeit und Nutzungsdauer der Software kann keine konkrete Aussage über Fehlerkosten abgeleitet werden, da andere Faktoren weitaus entscheidender sind.

ENTWICKLUNGS PROZESS



By Unknown photographer - State Library of South Australia -- https://collections.slsa.sa.gov.au/resource/B+41640, CC0, https://commons.wikimedia.org/w/index.php?curid=70649225

MODELL FÜR DEN SOFTWAREENTWICKLUNGSPROZESS

- Beschreibt eine Reihe von Aktivitäten, die in einer bestimmten Reihenfolge und von bestimmten Rollen durchgeführt werden, um Software-Artefakte zu erstellen
  - Ein Mitglied eines Softwareentwicklungsprojekts kann eine oder mehrere Rollen übernehmen.
  - Rollenverteilung unter den Mitgliedern kann sich ebenfalls ändern

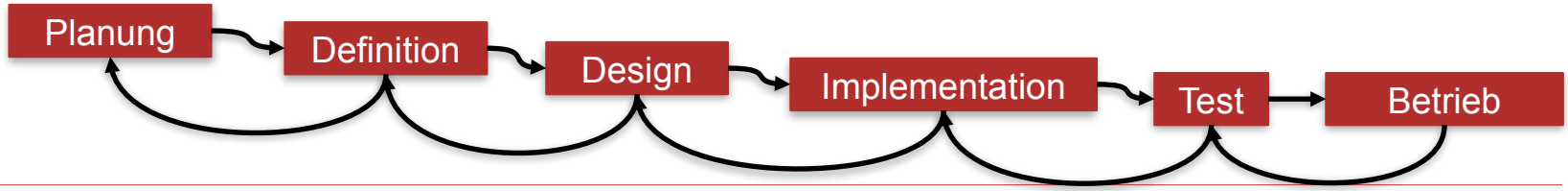




- Unterschiedliche Prozessmerkmale
  - Kleine vs. große Entwicklungsprojekte
  - Leichte vs. schwere Prozesse
  - Sequentielle vs. iterative Prozesse
  - Inkrementelle vs. evolutionäre Prozesse

WASSERFALLMODELL

- Jede Entwicklungsmaßnahme wird so umfassend wie möglich durchgeführt
  - Entwicklungsaktivitäten werden streng sequenziell durchgeführt
  - Bevor die nächste Aktivität beginnt, müssen die Ergebnisse der vorherigen Aktivität freigegeben werden.
  - In der Regel darf nur die unmittelbar vorhergehende Tätigkeit wieder aufgenommen werden
- Nachteile
  - Lange Verzögerung zwischen der Anforderungsanalyse und der Produktauslieferung
  - Anforderungen werden sehr früh festgelegt obwohl diese sich im Laufe der folgenden Entwicklungsaktivitäten ändern



MODELLE FÜR AGILE SOFTWAREENTWICKLUNGSPROZESSE

- Besonderer Schwerpunkt auf Projekten mit sich häufig ändernden und neu auftretenden Anforderungen
- Dies wird unter anderem erreicht durch
  - Enge Zusammenarbeit und Einbindung des Kunden
  - Entwicklung in kleinen Teams
  - Häufig werden Tests für ein Inkrement entworfen, bevor dieses entwickelt wird (Test-First-Ansatz)
  - Sehr kurze Entwicklungszyklen für kleine Produktinkremente, die sofort ausgeliefert werden können
  - Ausführbarer Code wird einer umfassenden Dokumentation vorgezogen

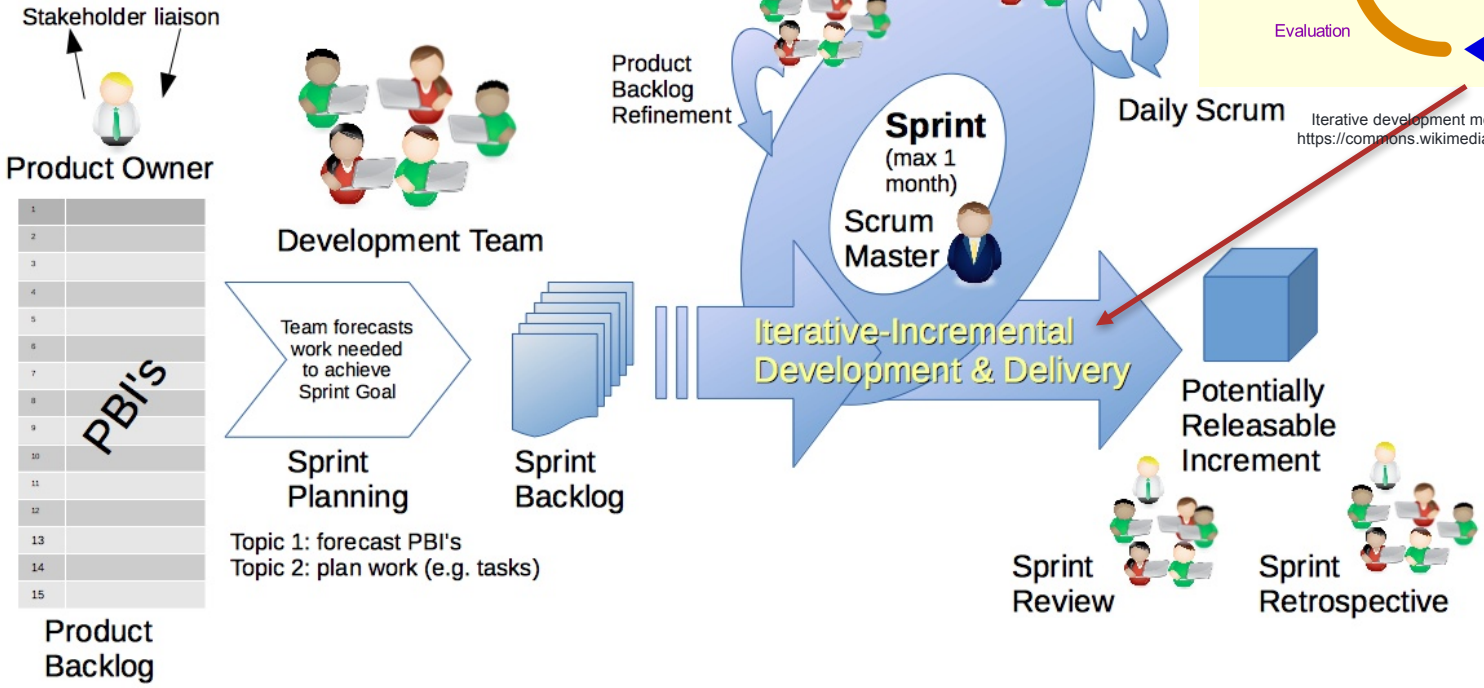
VOR- UND NACHTEILE DER AGILEN ENTWICKLUNG

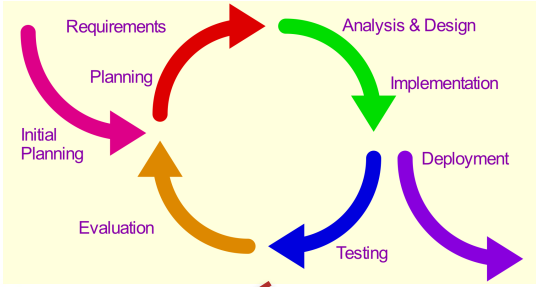
- Vorteile
  - Risiko minimieren, ein Produkt zu entwickeln, das die Kundenanforderungen nicht erfüllt
  - Minimierung des Risikos unnötiger Entwicklungsarbeit aufgrund geänderter Anforderungen
- Nachteile
  - Aufgrund der minimalen Dokumentation ist der vorhandene Quellcode die wichtigste Informationsquelle für eventuell erforderliche Wartungsarbeiten.
  - Der Quellcode spiegelt jedoch möglicherweise nicht direkt die zugrunde liegenden funktionalen Anforderungen wider.

SCRUM

- Ein agiles Prozessmodell zur Softwareentwicklung, das als „Scrum-Framework“ bezeichnet wird
  - Leichtgewichtig
  - Iterativ und inkrementell
  - Testorientiert
  - Anpassungsfähig an neue und sich ändernde Anforderungen
- Heute ist Scrum (in mehr oder weniger variantenreicher Form) ein weit verbreitetes Prozessmodell für die Softwareentwicklung (und in anderen Bereichen).
  - Es steht ein umfassendes Zertifizierungsprogramm zur Verfügung

SCRUM FRAMEWORK





By Aflafla1 – Iterative development model V2.jpg, User:Westerhoff, CC0, https://commons.wikimedia.org/w/index.php?curid=34159246

By Dr ian mitchell - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=44894952



GOOD TO KNOW

By Werner Huth - Published by Werner Huth, Public Domain, https://commons.wikimedia.org/wiki/File:Update16MioC-E.png

WICHTIGE HINWEISE (1)

- Ziel bei der Entwicklung und Einführung von Softwareprodukten ist es, Prozesse effizienter und effektiver zu gestalten
- Die Benutzung der Software, sollte den Prozess ändern
  - Die Auswirkungen dieser Veränderungen sind nicht immer absehbar
  - Dies kann zu neuen oder geänderten Anforderungen führen, den sogenannten „emergenten Anforderungen“
- In der Regel sollte die Einführung einer neuen Software zusätzliche Vorteile mit sich bringen, z. B.
  - Neue Dienstleistungen/Services
  - Vereinfachte Prozesse
  - Bereitstellung von mehr und relevanten Informationen

WICHTIGE HINWEISE (2)

- Ein Softwareprodukt unterstützt Prozesse, die von den Bedingungen abhängen, unter denen diese Prozesse ausgeführt werden.
  - Je nach Eingabedaten, Benutzerrolle, Systemeinstellung oder Umgebung kann die Software ihr Verhalten anpassen.
- Veränderungen auf dem Markt, in der Organisation und in der Strategie können Prozesse mehr oder weniger schnell verändern
  - Lange Verzögerung zwischen der Erfassung der Anforderungen und der Fertigstellung des Softwareprodukts
  - → Die Prozesse des Kunden haben sich möglicherweise bereits erheblich verändert
  - → Das Softwareprodukt entspricht nicht mehr den aktuellen Anforderungen

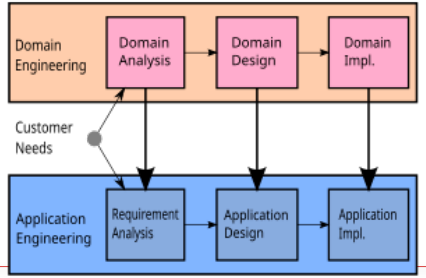
WICHTIGE HINWEISE (3)

- Manchmal wird ein Softwareprodukt für innovative Prozesse, Produkte oder Dienstleistungen entwickelt
- Innovationen bedeuten, dass in der Regel keine bisherigen Erfahrungen vorliegen.
  - Die entsprechenden Anforderungen müssen noch ausgearbeitet bzw. „konzipiert“ werden.
  - → Es können Anforderungen implementiert werden, die nicht ausreichend validiert wurden,
  - → Könnte sich später als falsch oder irrelevant erweisen

    → mangelhafter Prozess, mangelhaftes Produkt oder mangelhafte Dienstleistung

WICHTIGE HINWEISE (4)

- Softwareentwickler und Kunden/Nutzer sprechen in der Regel unterschiedliche Sprachen.
- Mehr oder weniger formalisierte Modelle von Geschäftsanforderungen und technischen Umsetzungen eignen sich kaum für die Kommunikation mit dem Nutzer
  - Quellprogramme sind für diesen Zweck noch weniger geeignet
- In natürlicher Sprache formulierte Anwendungsszenarien, Prototypen, Pilotsysteme oder Teilsysteme sind nach wie vor die besten Mittel, um mit dem Nutzer zu kommunizieren
  - Der Endverbraucher erkennt die Passform eines Produkts, wenn er es sieht.



By Metthew P. Del Buono, Under CC License, https://commons.wikimedia.org/wiki/File:Domain_Engineering_vs_Application_Engineering.svg

WICHTIGE HINWEISE (5)

- Der Kommunikationsaufwand, d. h. die Anzahl der Kommunikationsbeziehungen, steigt mit der Teamgröße

$$k = \frac{n \cdot (n-1)}{2}$$

k = Kommunikationsbeziehungen,
n = Anzahl der Teammitglieder

  - 2 Personen: 1 Kommunikationsbeziehung
  - 3 Personen: 3 Kommunikationsbeziehungen
  - 4 Personen: 6 Kommunikationsbeziehungen
- Um die Anzahl der Kommunikationsbeziehungen zu reduzieren, sollte ein großes Projekt in Teilprojekte unterteilt werden.
  - Im Idealfall besteht eine Entsprechung zwischen Produktteilen und Teilprojekten
  - Die Ermittlung geeigneter Teile eines Produkts, d. h. von Modulen, sollte sich an funktionalen Anforderungen oder technischen Einschränkungen orientieren

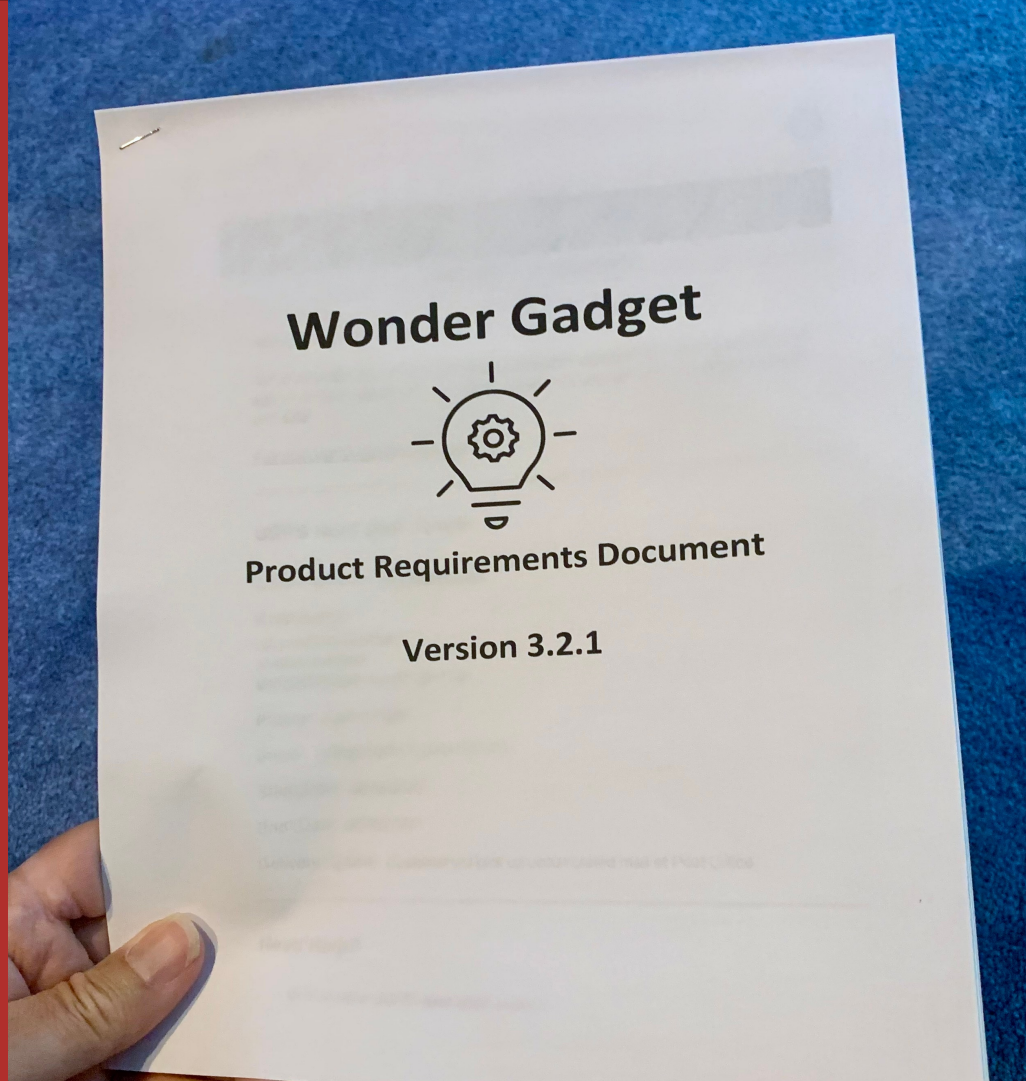
WICHTIGE HINWEISE (6)

- Die Aktivitäten von Teilprojekten lassen sich nicht beliebig parallelisieren
  - Insbesondere bei Innovationen erhöht dies das Risiko, dass Anforderungen nicht ausreichend validiert wurden oder dass Anforderungen fehlen, unvollständig oder fehlerhaft sind.
  - Die Integration zahlreicher Teilprodukte in einer späten Phase (Big Bang) kann viele Überraschungen bzw. Probleme mit sich bringen.
- Bei großen Projekten ist sehr wichtig, ein gutes Gleichgewicht zwischen parallel und nacheinander ablaufenden Teilprojekten zu finden.
- Frühzeitiges Feedback von Kunden und Nutzern sollte nach Möglichkeit berücksichtigt werden

AUSWIRKUNGEN

- Als Kunde/Nutzer sollten Sie in der Lage sein, Ihre Anforderungen zu formulieren
  - Zumindest die Kern- oder Grundanforderungen müssen genau beschrieben werden
- Die Anforderungen sollten in einem Lastenheft dokumentiert werden.
  - Es wird zu einem integralen Bestandteil des Vertrags
  - Es schützt sowohl Nutzer/Kunden als auch Auftragnehmer/Entwickler
  - Sollten sich die Anforderungen ändern, muss dies berücksichtigt werden, d. h., das Lastenheft und der Vertrag müssen entsprechend angepasst werden.
- Die technischen Daten werden durch ein Glossar ergänzt, in dem alle wichtigen Begriffe zur Beschreibung des Produkts definiert sind.
- Als Nutzer/Kunde sollten Sie in der Lage sein, mit dem Auftragnehmer/Entwickler ein für das Projekt geeignetes Prozessmodell zu besprechen.

SPEZIFIKATION & ANFORDERUNGEN



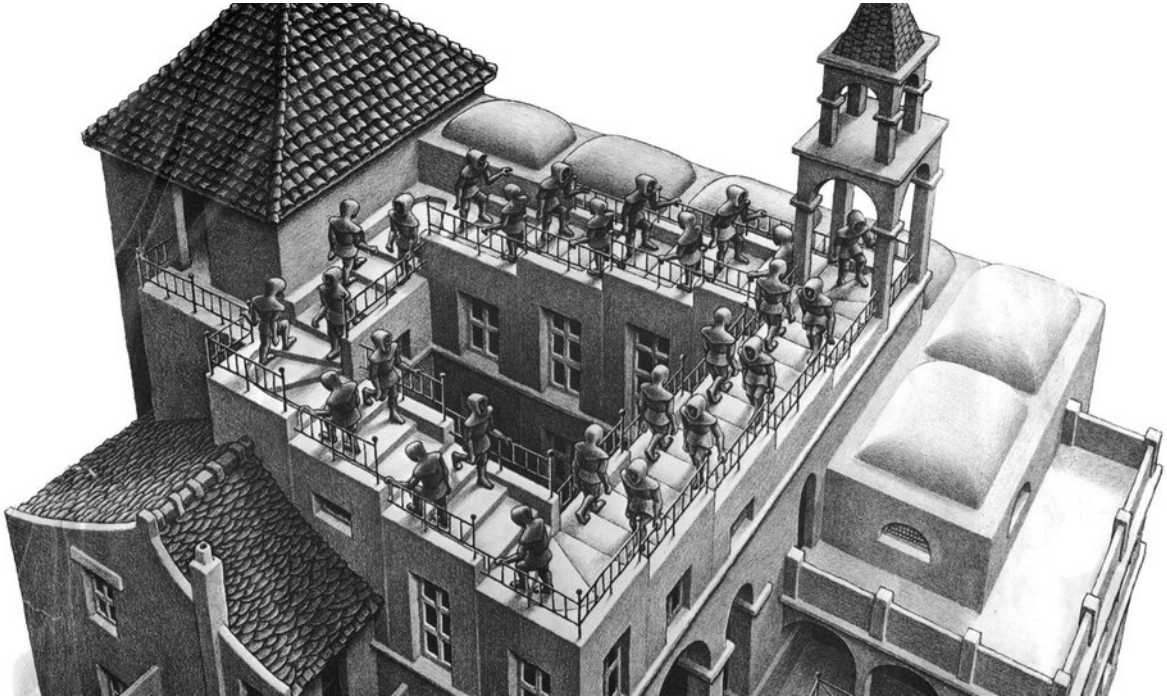
By Lboeaumont, Creative Commons
https://commons.wikimedia.org/wiki/File:Product_Requirements_Document.jpg

LASTENHEFT

- Deckt die grundlegenden funktionalen Anforderungen des Softwareprodukts ab
- Die Zielgruppe sind Auftraggeber und Auftragnehmer
- Mögliche Struktur
  - Zweck: Was ist der Zweck des Produkts?
  - Produktverwendung: Anwendungsbereiche und Zielgruppe des Produkts
  - Produktübersicht: Betriebsumgebung des Produkts
  - Produktfunktionen: Welche typischen Arbeitsabläufe gibt es? Welche funktionalen Anforderungen bestehen?
  - Produktdaten: Welche Daten müssen dauerhaft und in welchem Umfang gespeichert werden?
  - Produktdienstleistungen: Welche zeitlichen oder Genauigkeitsbeschränkungen gelten für Produktmerkmale und -daten?
  - Qualitätsanforderungen: Welche Qualitätsanforderungen weichen vom Unternehmensstandard ab, z. B. in Bezug auf Zuverlässigkeit, Benutzerfreundlichkeit und Effizienz?
  - Zusatzfunktionen: Zusatzfunktionen und spezielle Anforderungen wie Spracheingabe

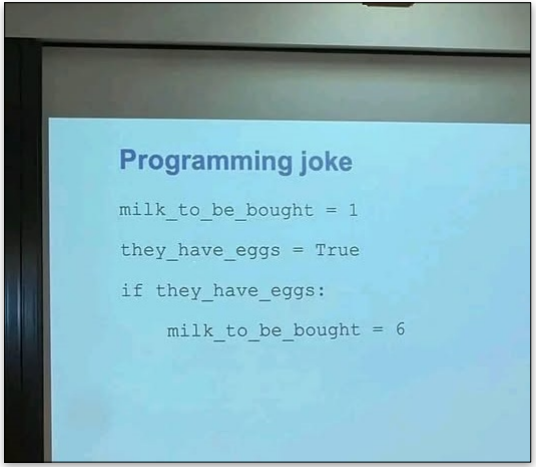
(Balzert und Balzert. Software-Management, Software-Qualitätssicherung, Unternehmensmodellierung.)

WARUM ANFORDERUNGEN?



MC Escher, section of "Ascending and Descending, 1960

ANALYSEFEHLER – WAS KÖNNTE SCHIEFGEHEN?



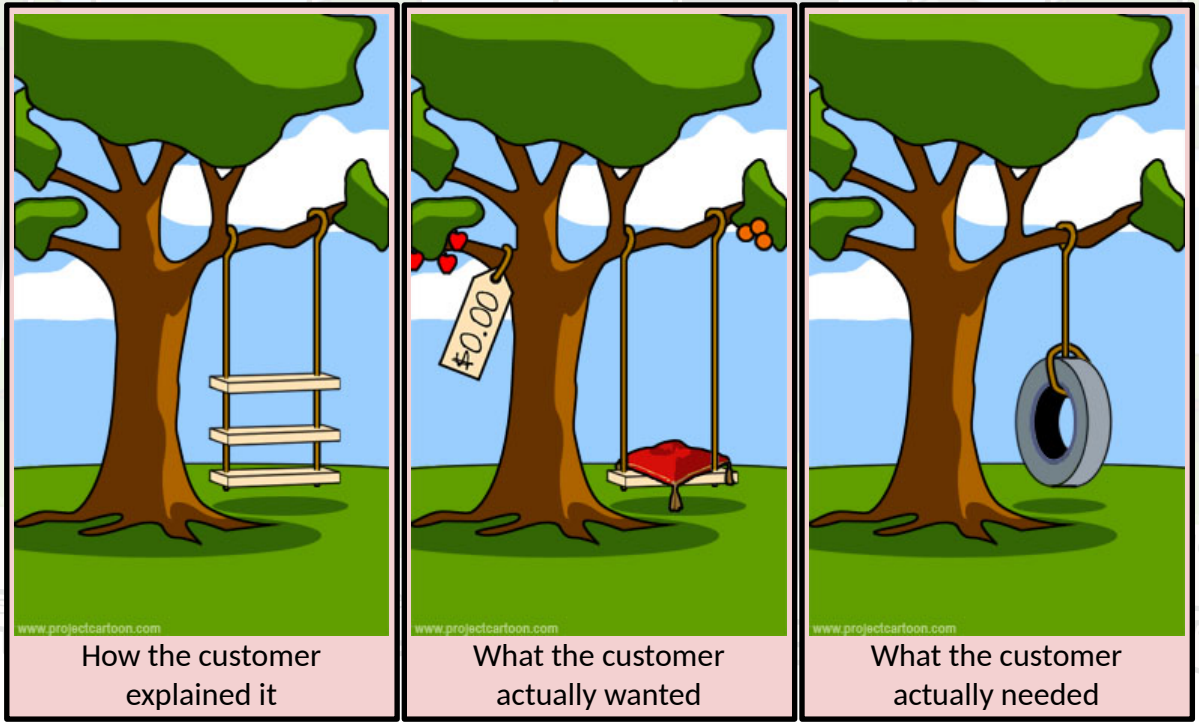
My wife said: „Please go to the store and buy a carton of milk and if they have eggs, get six.“

I came back with 6 cartons of milk.

She said: „Why in the hell did you buy six cartons of milk?“

I said: „They had eggs“

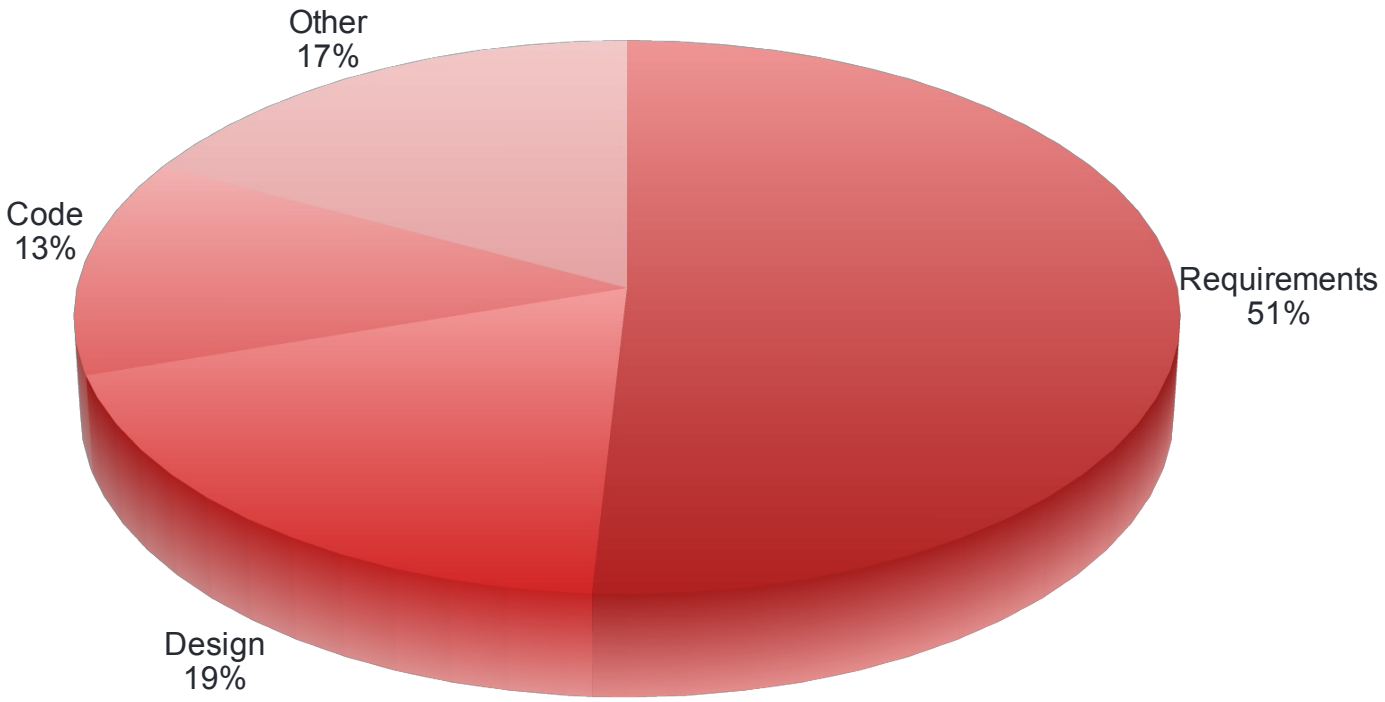
Durch ein gezieltes und korrektes Anforderungsmanagement lassen sich Abweichungen minimieren.



How the customer explained it | What the customer actually wanted | What the customer actually needed

http://projectcartoon.com

Fehlerverteilung in der Softwareentwicklung



Software Quality Through Requirement and Design, Massood Towhidnejad, Embry-Riddle Aeronautical University, 2006

DEFINITION: ANFORDERUNGSANALYSE (IREB*)

- Das Anforderungsmanagement ist ein **kooperativer, iterativer und inkrementeller** Prozess, der darauf abzielt, sicherzustellen, dass:
  - **Alle relevanten Anforderungen sind bekannt**, und die Softwareentwickler sollten wissen, **wie detailliert** sie arbeiten müssen.
  - Die **beteiligten Akteure erzielen eine ausreichende Einigung** über die bekannten Anforderungen
  - Alle Anforderungen sind **gemäß den Dokumentationsvorschriften dokumentiert** oder **gemäß den Spezifikationsvorschriften festgelegt**.

*IREB: International Requirements Engineering Board

(Klaus Pohl, Requirements Engineering, 2010)

ANFORDERUNGSDEFINITION GEMÄß IEEE UND IREB

- Eine Anforderung ist
  1. Eine Voraussetzung oder Fähigkeit, die ein Nutzer (eine Person oder ein System) benötigt, um ein Problem zu lösen oder ein Ziel zu erreichen
  2. Eine Anforderung oder Eigenschaft, die ein System oder Teilsystem erfüllen bzw. aufweisen muss, um den Bestimmungen eines Vertrags, einer Norm, einer Spezifikation oder eines anderen formal festgelegten Dokuments zu entsprechen
  3. Eine dokumentierte Darstellung eines Zustands oder einer Eigenschaft gemäß (1) oder (2)
- Oder vereinfacht ausgedrückt
  - Eine Anforderung ist eine Aussage über eine Eigenschaft oder Leistung eines Produkts, eines Prozesses oder der an dem Prozess bet

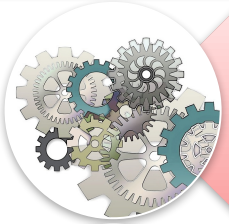
*IEEE: Institute of Electrical and Electronics Engineers

(Rupp & Die SOPHISTen, 2009, pp. 12-13)

ANFORDERUNGEN (1/3)



Eine Anforderung ist jedoch nicht nur eine Anfrage nach einer Leistung, die am Ende der Softwareentwicklung erbracht wurde.



Es gibt Anforderungen in unterschiedlichem Detaillierungsgrad und in verschiedenen Klassifizierungen



Es gibt Anforderungen an Software, Hardware, Testfälle, Handbücher, Protokolle, die Sicherheitsüberprüfung der Mitarbeiter sowie bestimmter Qualifikationen der Entwickler

(Rupp & Die SOPHISTen, 2009, p. 14)

ANFORDERUNGEN (2/3)



Die Anforderungen müssen das gemeinsame Verständnis und den **gemeinsamen Wissensstand** der Beteiligten widerspiegeln.



Anforderungen bilden die Grundlage für Ausschreibungen, Vertragsentwürfe und Systemzulassungen

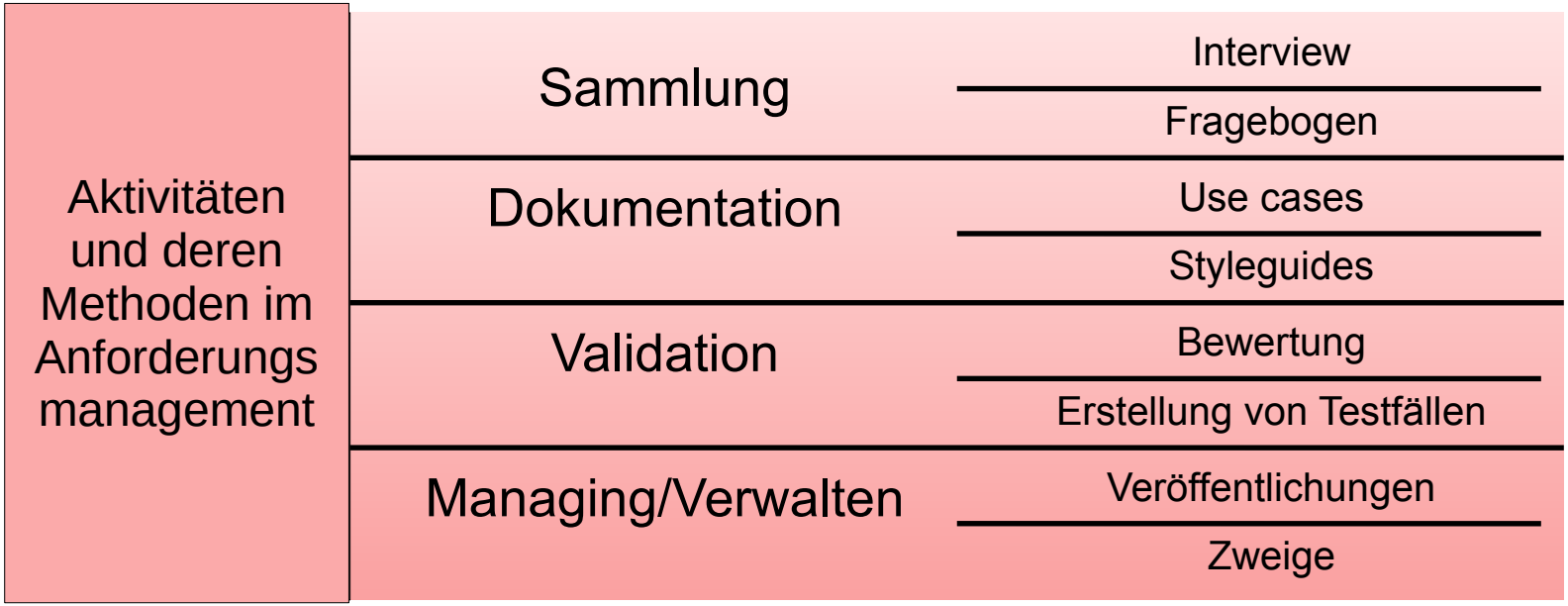
Vergleich zwischen IST-Zustand (wie reagiert das System tatsächlich) und SOLL-Zustand (was ist in den Anforderungen festgelegt)

Eine genaue Definition der Anforderungen ist unerlässlich*.

*allgemein gesprochen…

(Rupp & Die SOPHISTen, 2009, p. 16)

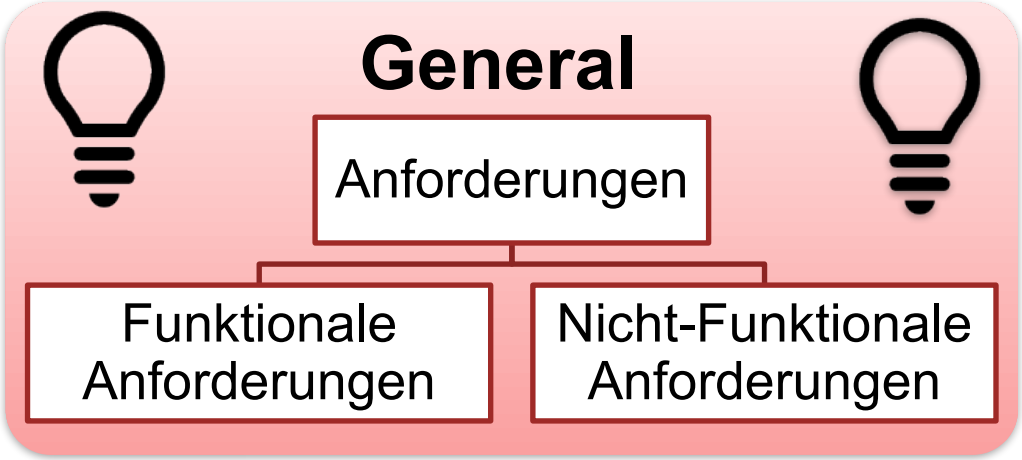
ANFORDERUNGEN (3/3)



(Rupp & Die SOPHISTen, 2009, p. 14)

KLASSIFIZIERUNG VON ANFORDERUNGEN (1/2)

- IREB
  - Funktionale Anforderungen
  - Qualitätsanforderungen
  - **Randbedingungen**
- Oder
  - Funktionale Anforderungen
  - Technische Anforderungen
  - Qualitätsanforderungen
  - Anforderungen an die Benutzeroberfläche
  - Anforderungen an weitere Lieferkomponenten
  - Anforderungen an die durchzuführenden Tätigkeiten und rechtliche sowie vertragliche Anforderungen



(Rupp & Die SOPHISTen, 2009, p. 15)

KLASSIFIZIERUNG VON ANFORDERUNGEN (2/2)

Funktionale Anforderungen	Nicht-Funktionale Anforderungen
Beschreiben Sie die gewünschten Funktionen “Das Produkt **muss** in der Lage sein, Kaffee zuzubereiten.“ "Das Produkt **sollte** über einen Ein-/Aus-Schalter verfügen."	**Beschreiben Sie die Anforderungen an „Qualität“** „Der Kaffee **soll** innerhalb von 30 Sekunden fertig sein.“ „Das Produkt **sollte** rund um die Uhr laufen.“
Der Kunde wird hauptsächlich funktionale Anforderungen formulieren	Der Kunde wird wahrscheinlich nur dann an nicht-funktionale Anforderungen denken, wenn er in der Vergangenheit schlechte Erfahrungen gemacht hat (z. B. schlechte Qualität, geringe Verfügbarkeit).
Das ist von Produkt zu Produkt unterschiedlich.	Kann bei vielen Produkten des Unternehmens ähnlich oder sogar standardisiert sein, z. B. ein einheitliches Erscheinungsbild
Haben in der Regel nur „lokale“ Auswirkungen auf das Produkt	Haben in der Regel „globale“/weitreichende Auswirkungen auf das Produkt

KLASSIFIZIERUNG NACH HAFTBARKEIT

Haftbarkeit	Deutsches Schlüsselwort	Englisches Schlüsselwort	Rechtswirksamkeit
Pflicht	**muss**	**shall**	Rechtlich bindend – muss erfüllt werden
Wunsch	**sollte**	**should**	Nicht rechtsverbindlich – gibt jedoch die Absicht wieder
Zweck	**wird**	**will**	Weist auf künftige Entwicklungen hin, die berücksichtigt werden sollen



Or may, can



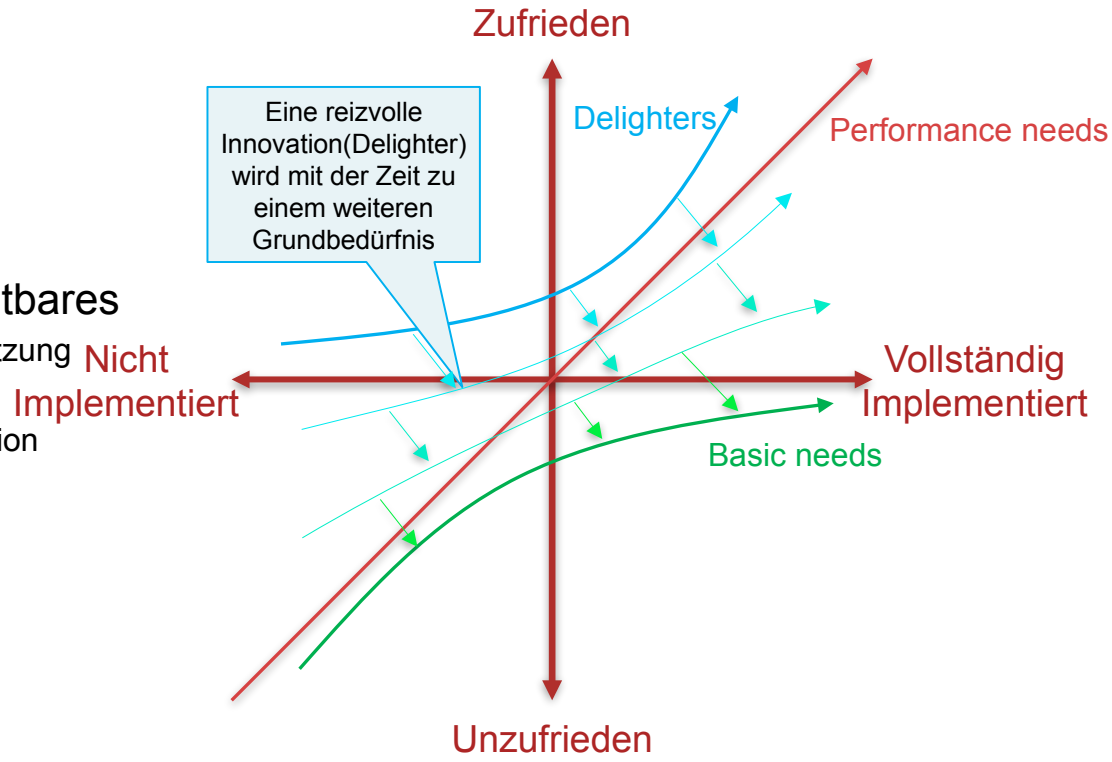
Definieren Sie gegebenenfalls auch Nicht-Anforderungen.

Es gibt kein 100-prozentiges Richtig oder Falsch, sondern sollte im Vertrag festgesetzt werden.

(Rupp & Die SOPHISTen, 2014, pp. 18-19)

KANO MODEL (1/2)

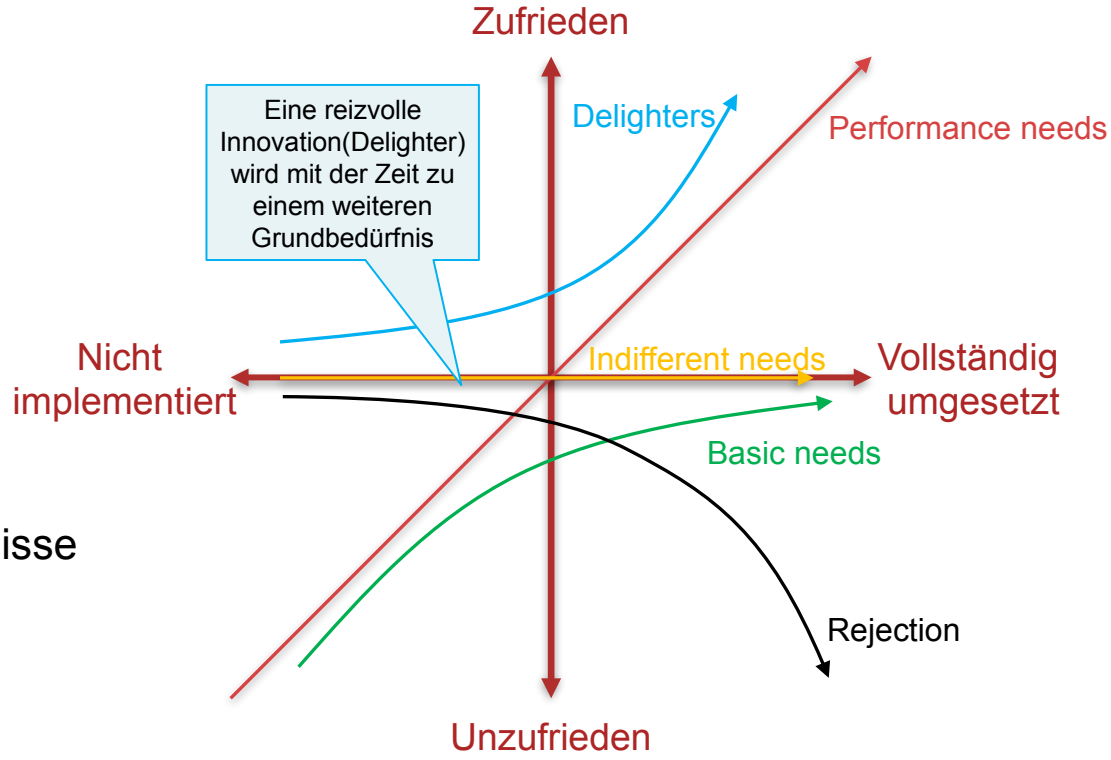
- Ist eine Theorie zur Produktentwicklung und Kundenzufriedenheit
  - Grundbedürfnisse / Unverzichtbares
    - Der Kunde erwartet deren Umsetzung
    - Gut gemacht → neutral
    - Schlecht gemacht → dissatisfaction
  - Leistungsanforderungen
    - Kunden nimmt es bewusst war
    - Gut gemacht → Zufriedenheit
    - Nicht/nur unzureichend erfüllt → Unzufriedenheit



(Rupp & Die SOPHISTen, 2014, pp. 95-97; Page "Kano-Modell". In: Wikipedia, The Free Encyclopedia https://de.wikipedia.org/w/index.php?title=Kano-Modell&oldid=180219516 (Retrieved: October 17, 2018))

KANO MODEL (2/2)

- Delighter / Begeisterer / Attraktiv
  - Der Kunde erwartet es nicht
  - Gut gemacht → Zufriedenheit
  - Schlecht gemacht → neutral
- Indifferente Needs/Bedürfnisse
  - Dem Kunden ist das egal
  - Gut gemacht → neutral
  - Schlecht gemacht → neutral
- Ablehnung / umgekehrte Bedürfnisse
  - Der Kunde möchte das nicht.
  - Umsetzung → Unzufriedenheit
  - Nicht umgesetzt → neutra



(Rupp & Die SOPHISTen, 2014, pp. 95-97; Page "Kano-Modell". In: Wikipedia, The Free Encyclopedia https://de.wikipedia.org/w/index.php?title=Kano-Modell&oldid=180219516 (Retrieved: October 17, 2018))

PROBLEME MIT ANFORDERUNGEN

Hauptprobleme bei der Anforderungsanalyse

- Unklare Ziele
- Hohe Komplexität
- Sprachbarrieren
- Mangelhafte Qualität
- Schwankende Anforderungen
- Informationsverlust

Anforderungen an Anforderungen

Woran erkenne ich, ob eine Anforderung gut formuliert ist?

- Vollständig
- Erforderlich
- Atomar
- Nachvollziehbar
- Technisch lösungsneutral
- Realisierbar
- Konsequent
- Eindeutig
- Prüfbar/

(Rupp & Die SOPHISTen, 2014, pp. 24-28)

UNKLAR FORMULIERTE ANFORDERUNG

Haftungsklausel formulieren
soll, sollte, darf, kann, wird

Vermeidung unbestimmter Quantoren
Falsch: irgendein, meistens, gewöhnlich, irgendwie
Besser: alle, jeder, und, immer, oder, keiner

Es wäre schön, wenn sich die Daten mit json2xls irgendwie ganz einfach nach Excel exportieren ließen.

Substantive vollständig angeben
Welche Daten?

Keine Lösungen angeben

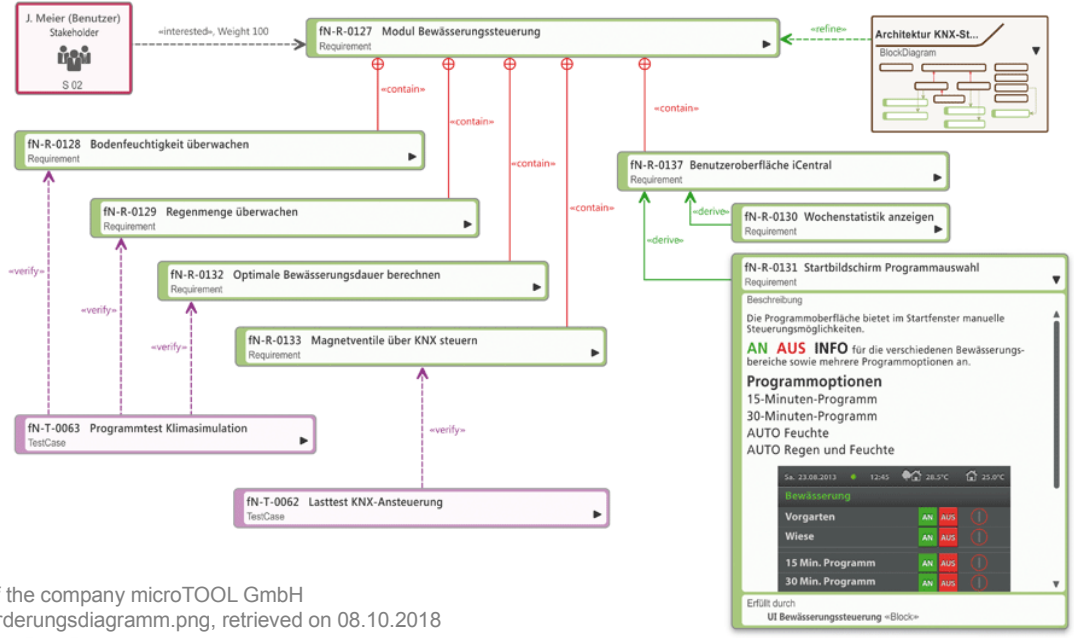
Umfassende Vergleiche und Steigerungen
Einfach nur, um im Vergleich zu was zu leisten?

(Belikan, 2007)

ANFORDERUNGSDIAGRAMM

Es bestehen Zusammenhänge zwischen verschiedenen Anforderungen.

- Insbesondere dann, wenn Anforderungen Spezifikationen anderer Anforderungen sind



Requirements diagram of the SysML of the company microTOOL GmbH
https://de.wikipedia.org/wiki/Datei:Anforderungsdiagramm.png, retrieved on 08.10.2018

5 SPEZIFIKATIONSSTUFEN

0

- Sehr abstrakte Anforderung
- Beschreibt den Gesamtumfang in wenigen Sätzen
- Zur Vermittlung eines Überblicks: Einleitung
- Enthält Absichtserklärungen, Ziele, Hauptmerkmale oder das „Leitbild“

Management – „Das große Ganze“

1

- Ermittlung der Teilschritte zur Anforderungsermittlung des Geschäftsprozesses
- Beschreibung anhand von Anwendungsfällen und Epics

Die Mitarbeiter der Abteilung müssen diese Anforderungen verstehen

2

- Kern der Systemanalyse
- Für Prozesse der Spezifikationsstufe 1 werden technische Abläufe definiert.
- Für jede der beteiligten Parteien ergeben sich Aufgaben.
- User Stories

Die Mitarbeiter der Abteilung müssen die Anforderungen der Prozesse verstehen, die sie betreffen

3

- Abschluss der technischen Spezifikationen
- Betrachten Sie die (Teil-)Prozesse auf Ebene 2 genauer und verfeinern Sie diese.

Die Mitarbeiter der Abteilung müssen die Anforderungen der Prozesse verstehen, die sie betreffen

4

- Berücksichtigt die Aufteilung des Systems in Software und Hardware oder andere Komponenten
- Anforderungen an diese Systemkomponenten
- Bietet technologische Lösungen für Anforderungen anderer Spezifikationsebenen

Für die Mitarbeiter der Abteilung sind sie nicht mehr relevant; der Architekt muss sie verstehen

(Rupp & Die SOPHISTen, 2014, pp. 40-41)

VORBEREITUNG

- Bevor Anforderungen festgelegt werden, muss eine Grundlage geschaffen werden.
  - Kenntnisse über den Kontext erwerben
  - Die Grenzen des zu entwickelnden Systems festlegen
  - Legen Sie die Ziele des Projekts fest
  - **Anforderungsquellen ermitteln**
    - **Stakeholders**
    - **Dokumente (z. B. Gesetzestexte)**
    - **Risikoanalyse**
    - **Produktideen**
    - **Bewährte Vorgehensweise**
    - **Andere Systeme (z. B. Vorgängersysteme oder Systeme von Mitbewerbern)**



(Rupp & Die SOPHISTen, 2014, p. 74)

BEISPIELE FÜR GESETZESTEXTE

STRAßENVERKEHRS-ZULASSUNGS-ORDNUNG (STVZO)

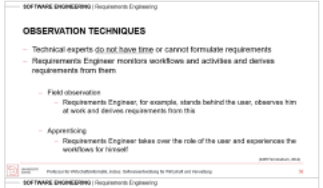
§ 53 SCHLUSSLEUCHTEN, BREMSLEUCHTEN, RÜCKSTRAHLER

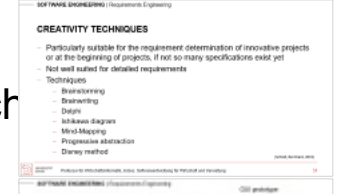
- (1) Kraftfahrzeuge und ihre Anhänger müssen hinten mit **zwei** ausreichend wirkenden Schlussleuchten für **rotes Licht** ausgerüstet sein. [..] Der **niedrigste Punkt der leuchtenden Fläche der Schlussleuchten darf nicht tiefer als 350 mm**, [..] und der **höchste Punkt der leuchtenden Fläche nicht höher als 1 500 mm**, [..]

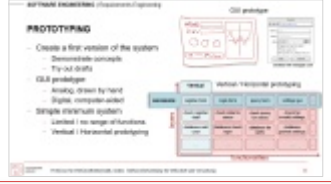
UNTERSUCHUNGSMETHODEN FÜR ANFORDERUNGEN

- Interviewtechniken
  - Interview
  - Fragebogen
- Beobachtungstechniken
  - Feldbeobachtung
  - Apprenticing/Lehrlingsausbildung
- Kreativitätstechniken
  - Brainstorming (schriftlich/mündlich
  - Workshop
- Prototyping
  - GUI-prototype











INTERVIEWTECHNIKEN

- Die Beteiligten **müssen Zeit haben** und die Möglichkeit erhalten, ihre Anforderungen zu äußern
- Für alle Arten von Anforderungen geeignet
- Interview
  - Fragen vorbereiten, stellen und direkt protokollieren; bei Bedarf zusätzlich einen Protokollierer einsetzen
- Fragebogen

BEOBACHTUNGSTECHNIKEN

- Wenn Fachleute keine Zeit haben oder sind nicht in der Lage sind, Anforderungen zu formulieren
- Der Requirements Engineer überwacht Arbeitsabläufe und Aktivitäten und leitet daraus Anforderungen ab

  - Feldbeobachtung
    - Der Anforderungsingenieur steht beispielsweise hinter dem Benutzer, beobachtet ihn bei der Arbeit und leitet daraus Anforderungen ab

  - Apprenticing/Lehrlingsausbildung
    - Der Requirements Engineer schlüpft in die Rolle des Anwenders und erlebt die Arbeitsabläufe selbst

(IUBH Fernstudium, 2014)

KREATIVITÄTSTECHNIKEN

- Besonders geeignet für die Anforderungsermittlung bei innovativen Projekten oder zu Beginn von Projekten, wenn noch nicht so viele Spezifikationen vorliegen
- Nicht besonders gut geeignet für detaillierte Anforderungen
- Techniques
  - Brainstorming
  - Brainwriting
  - Delphi
  - Ishikawa diagram
  - Mind-Mapping
  - Progressive abstraction
  - Disney method

(Schloß, Bernhard, 2015)

KREATIVITÄTSTECHNIKEN

- Besonders geeignet für die Anforderungsermittlung bei innovativen Projekten oder zu Beginn von Projekten, wenn noch nicht so viele Spezifikationen vorliegen
- Nicht besonders gut geeignet für detaillierte Anforderungen
- Techniques
  - **Brainstorming**
  - Brainwriting
  - Delphi
  - Ishikawa diagram
  - Mind-Mapping
  - Progressive abstraction
  - Disney method

- Eine Gruppe aus einer beliebigen Anzahl von Personen
  - Experten, Mitarbeiter, Laien …
- Ideen sammeln
  - Keine Kritik, Bewertung oder Beurteilung der Ideen
  - Keine „Killer-Sätze“
  - Seid kreativ
- Ideen sortieren und bewerten

Page “Brainstorming". In: Wikipedia, The Free Encyclopedia
https://de.wikipedia.org/w/index.php?title=Brainstorming&oldid=176567551 (Retrieved: October 10, 2018)

KREATIVITÄTSTECHNIKEN

- Besonders geeignet für die Anforderungsermittlung bei innovativen Projekten oder zu Beginn von Projekten, wenn noch nicht so viele Spezifikationen vorliegen
- Nicht besonders gut geeignet für detaillierte Anforderungen
- Techniques
  - Brainstorming
  - **Brainwriting**
  - Delphi
  - Ishikawa diagram
  - Mind-Mapping
  - Progressive abstraction
  - Disney method

- Ähnlich wie beim Brainstorming
- Jeder Teilnehmer schreibt seine Ideen auf, gegebenenfalls anonym
- Mehrere Runden: Ideen aufschreiben, sammeln und kritikfrei präsentieren
- Anschließend die Ergebnisse im Detail auswerten

Page "Brainwriting". In: Wikipedia, The Free Encyclopedia
https://de.wikipedia.org/w/index.php?title=Brainwriting&oldid=180833910 (Retrieved: October 10, 2018)

KREATIVITÄTSTECHNIKEN

- Besonders geeignet für die Anforderungsermittlung bei innovativen Projekten oder zu Beginn von Projekten, wenn noch nicht so viele Spezifikationen vorliegen
- Nicht besonders gut geeignet für detaillierte Anforderungen
- Techniques
  - Brainstorming
  - Brainwriting
  - **Delphi**
  - Ishikawa diagram
  - Mind-Mapping
  - Progressive abstraction
  - Disney method

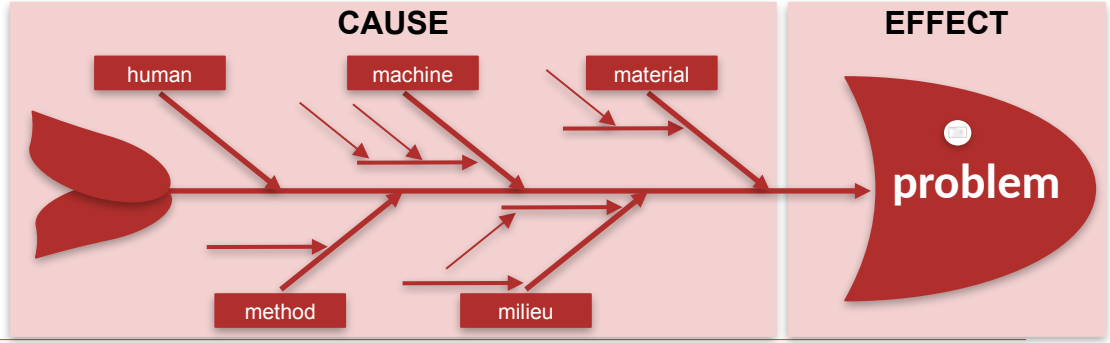
- Mehrstufige Expertenbefragung
- Experten bewerten vorgegebene Thesen
  - Die Schätzungen werden anonym und mit Begründung dargestellt.
  - Nach mehreren Durchläufen nähern sich die Einschätzungen immer mehr an.

Page "Delphi-Methode". In: Wikipedia, The Free Encyclopedia
https://de.wikipedia.org/w/index.php?title=Delphi-Methode&oldid=179029940 (Retrieved: October 10, 2018)

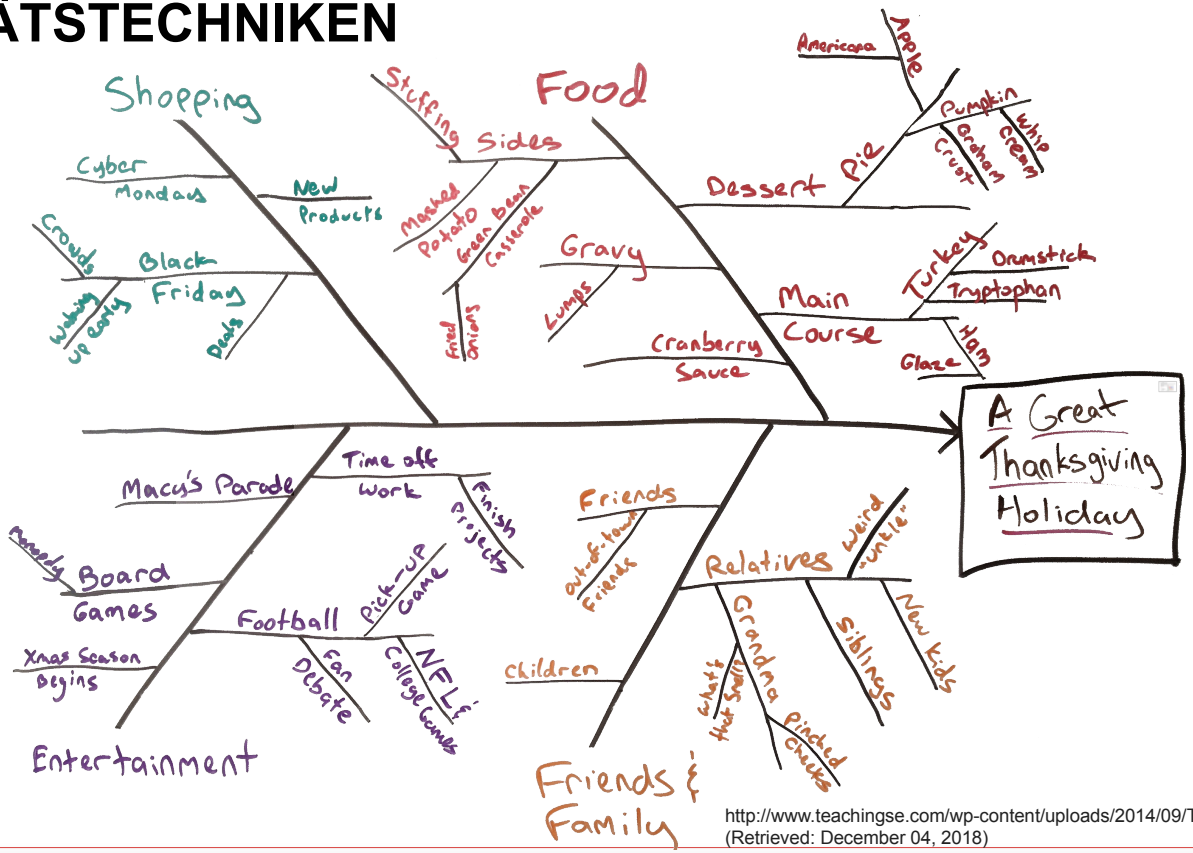
KREATIVITÄTSTECHNIKEN

- Besonders geeignet für die Anforderungsermittlung bei innovativen Projekten oder zu Beginn von Projekten, wenn noch nicht so viele Spezifikationen vorliegen
- Nicht besonders gut geeignet für detaillierte Anforderungen
- Techniques
  - Brainstorming
  - Brainwriting
  - Delphi
  - **Ishikawa diagram**
  - Mind-Mapping
  - Progressive abstraction
  - Disney method

- Untersucht Ursachen in verschiedenen Dimensionen
  - Mensch, Maschine, Material, Umfeld, Management, Methode, Messung, Geld…



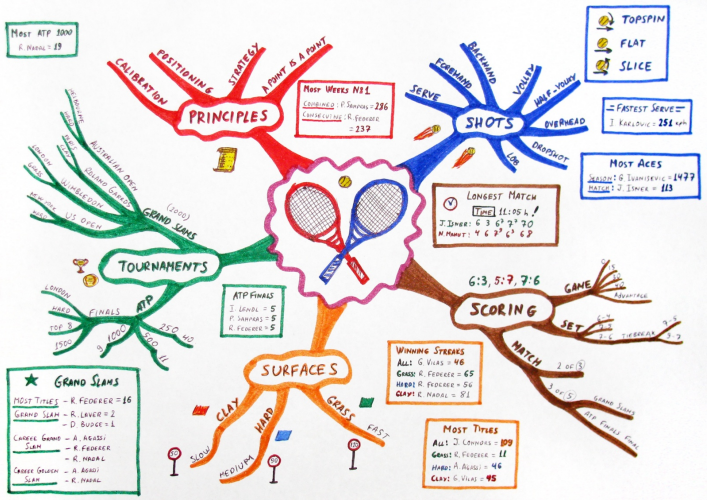
KREATIVITÄTSTECHNIKEN



http://www.teachingse.com/wp-content/uploads/2014/09/ThanksgivingFishbone1.gif
(Retrieved: December 04, 2018)

KREATIVITÄTSTECHNIKEN

- Besonders geeignet für die Anforderungsermittlung bei innovativen Projekten oder zu Beginn von Projekten, wenn noch nicht so viele Spezifikationen vorliegen
- Nicht besonders gut geeignet für detaillierte Anforderungen
- Techniques
  - Brainstorming
  - Brainwriting
  - Delphi
  - Ishikawa diagram
  - **Mind-Mapping**
  - Progressive abstraction
  - Disney method



https://upload.wikimedia.org/wikipedia/commons/1/1a/Tennis-mindmap.png (Retrieved: October 10, 2018)

KREATIVITÄTSTECHNIKEN

- Besonders geeignet für die Anforderungsermittlung bei innovativen Projekten oder zu Beginn von Projekten, wenn noch nicht so viele Spezifikationen vorliegen
- Nicht besonders gut geeignet für detaillierte Anforderungen
- Techniques
  - Brainstorming
  - Brainwriting
  - Delphi
  - Ishikawa diagram
  - Mind-Mapping
  - **Progressive abstraction**
  - Disney method

- Durch eine veränderte Perspektive auf das Projekt lassen sich neue Lösungen finden
  - Finde das Problem auf der nächsthöheren Ebene
    - „Was ist denn schon dabei?“
    - „Wie sonst könnte das funktionieren?“
  - Wenn keine Lösung gefunden wird, weiter abstrahieren
- Paradoxe Intervention
  - „Was können wir tun, um das Problem noch zu verschlimmern?“

Page "Progressive Abstraktion". In: Wikipedia, The Free Encyclopedia https://de.wikipedia.org/w/index.php?title=Progressive_Abstraktion&oldid=169414231 (Retrieved: October 10, 2018)

KREATIVITÄTSTECHNIKEN

- Besonders geeignet für die Anforderungsermittlung bei innovativen Projekten oder zu Beginn von Projekten, wenn noch nicht so viele Spezifikationen vorliegen
- Nicht besonders gut geeignet für detaillierte Anforderungen
- Techniques
  - Brainstorming
  - Brainwriting
  - Delphi
  - Ishikawa diagram
  - Mind-Mapping
  - Progressive abstraction
  - **Disney method**

- "…there were actually three different Walts: the dreamer, the realist, and the spoiler"

  Robert B. Dilts

- Das Problem wird aus drei Blickwinkeln betrachtet
  - Der Träumer: subjektiv und enthusiastisch
  - Der Realist: pragmatisch-praktisch
  - Der Kritiker: suchen und prüfen
  - Kann von Einzelpersonen oder Gruppen genutzt werden

Page "Walt-Disney-Methode". In: Wikipedia, The Free Encyclopedia https://de.wikipedia.org/w/index.php?title=Walt-Disney-Methode&oldid=181458213 (Retrieved: October 10, 2018)

KREATIVITÄTSTECHNIKEN

- Besonders geeignet für die Anforderungsermittlung bei innovativen Projekten oder zu Beginn von Projekten, wenn noch nicht so viele Spezifikationen vorliegen
- Nicht besonders gut geeignet für detaillierte Anforderungen
- Techniques
  - Brainstorming
  - Brainwriting
  - Delphi
  - Ishikawa diagram
  - Mind-Mapping
  - Progressive abstraction
  - Disney method

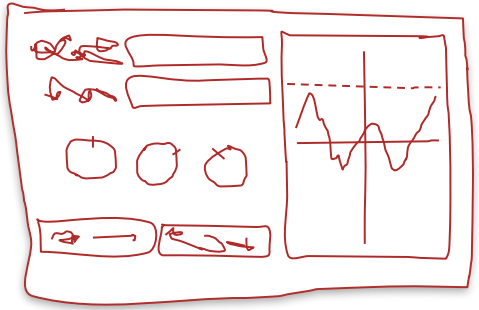
Design Thinking

(Schloß, Bernhard, 2015)

PROTOTYPING

- Eine erste Version des Systems erstellen
  - Konzepte veranschaulichen
  - Entwürfe ausprobieren
- GUI prototype
  - Analog, von Hand gezeichnet
  - Digital, computergestützt
- Einfaches Minimalsystem (MVP)
  - Eingeschränkter bzw. fehlender Funktionsumfang
  - Vertikales / horizontales Prototyping

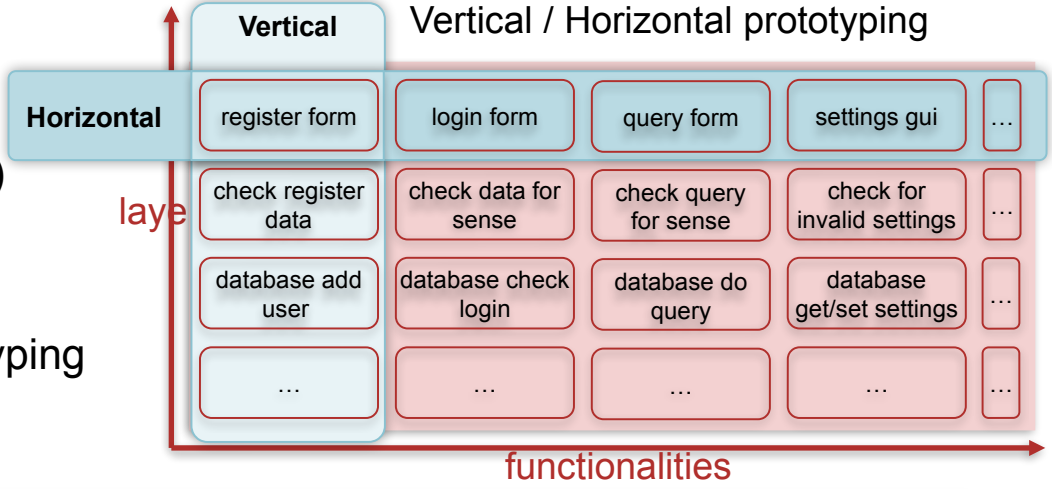
GUI prototype

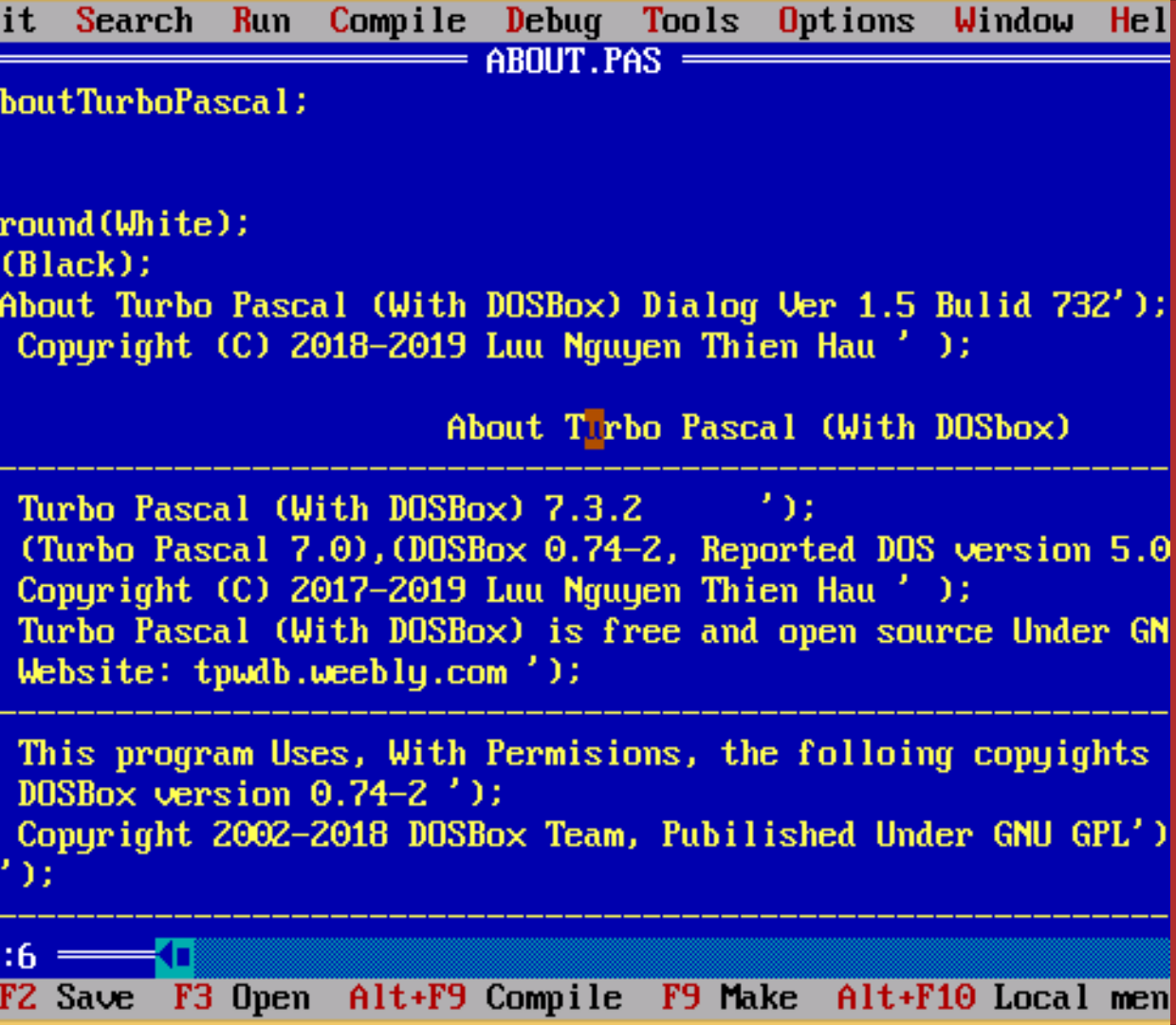




created with moqups.com

Vertical / Horizontal prototyping





PROGRAMMBEARBEITUNG UND -AUSFÜHRUNG

By Lưu Nguyễn Thiện Hậu - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=75160140, no longer online available

BEARBEITUNG EINES QUELLPROGRAMMS

- Ein Quellprogramm wird mit einem Editor eingegeben und bearbeitet.
  - Es gibt eigenständige Editoren, z. B. edit, edlin, vi, nano
  - Oder ein Editor ist Teil einer sogenannten integrierten Entwicklungsumgebung (Abkürzung: IDE), z. B. Visual Studio, Eclipse, Code::Blocks



- Das Beispielprogramm ist in der Programmiersprache C++ geschrieben und gibt „Hallo Welt“ aus.
- Die Bearbeitung erfolgt in einem Windows-Konsolenfenster mit dem Programm „edit“ (das nicht mehr zu Windows gehört).

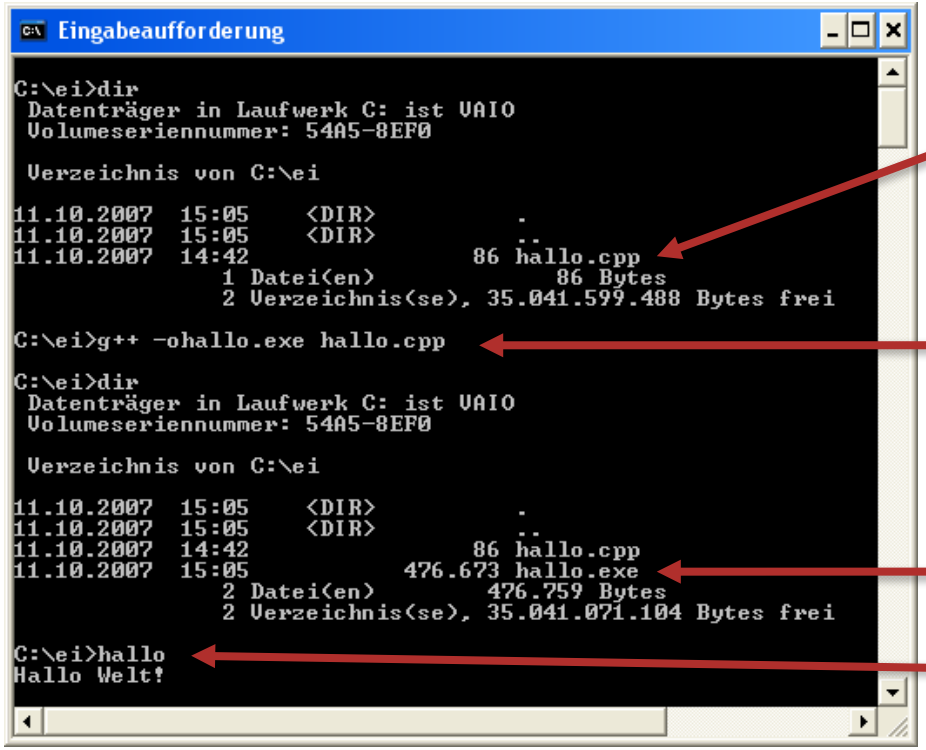
QUELLPROGRAMM VS. AUSFÜHRBARES PROGRAMM

- Quellcode wird in der Regel von einem Menschen in einer bestimmten Programmiersprache geschrieben.
  - Verfasst für andere Menschen, damit diese den Text lesen, verstehen, anpassen, pflegen oder Fehler korrigieren können
  - Entweder verarbeitet
    - Ein Compiler, der ein ausführbares Programm erzeugt
    - Ein Interpreter, der den Code Zeile für Zeile ausführt
- Ein ausführbares Programm wird auch als Binärprogramm bezeichnet.
  - Enthält Programme und Daten in binärer Form, die von einer CPU ausgeführt werden sollen

PROGRAMMAUSFÜHRUNG

- Zwei Möglichkeiten, ein Programm auszuführen
  - Ein Compiler übersetzt das Quellprogramm in ein ausführbares Programm (Binärprogramm).
    - Das ausführbare Programm kann gestartet werden, indem man beispielsweise seinen Namen in ein Konsolenfenster eingibt und die Eingabetaste drückt.
    - Das ausführbare Programm kann separat, d. h. ohne das zugehörige Quellprogramm
  - Das Quellprogramm wird in einen Interpreter geladen, der jede Zeile ad hoc übersetzt und ausführt
    - Es wird kein eigenständiges ausführbares Programm erstellt.
    - Um das Programm auszuführen, müssen sowohl der Quellcode als auch ein entsprechender Interpreter verfügbar sein.

BEISPIEL FÜR EIN KOMPILIERTES PROGRAMM



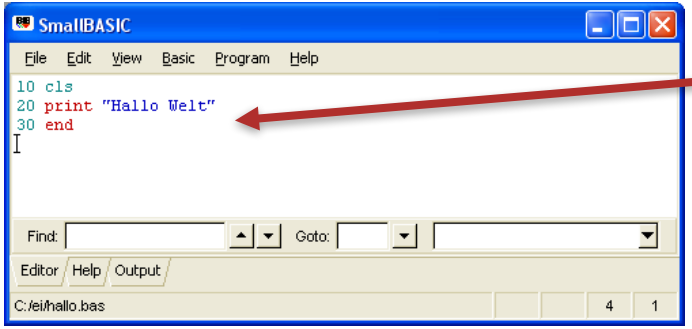
Quellprogramm

Aufruf des Compilers zur Erstellung des ausführbaren Programms

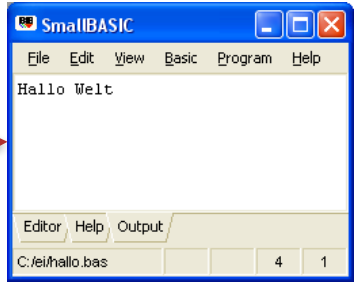
Ausführbares Programm

Aufrufen des ausführbaren Programms

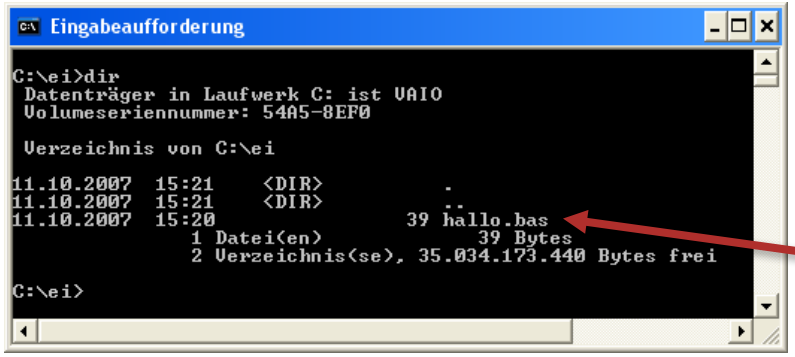
BEISPIEL FÜR EIN INTERPRETIERTES PROGRAMM



Quellcode im SmallBASIC-Editor

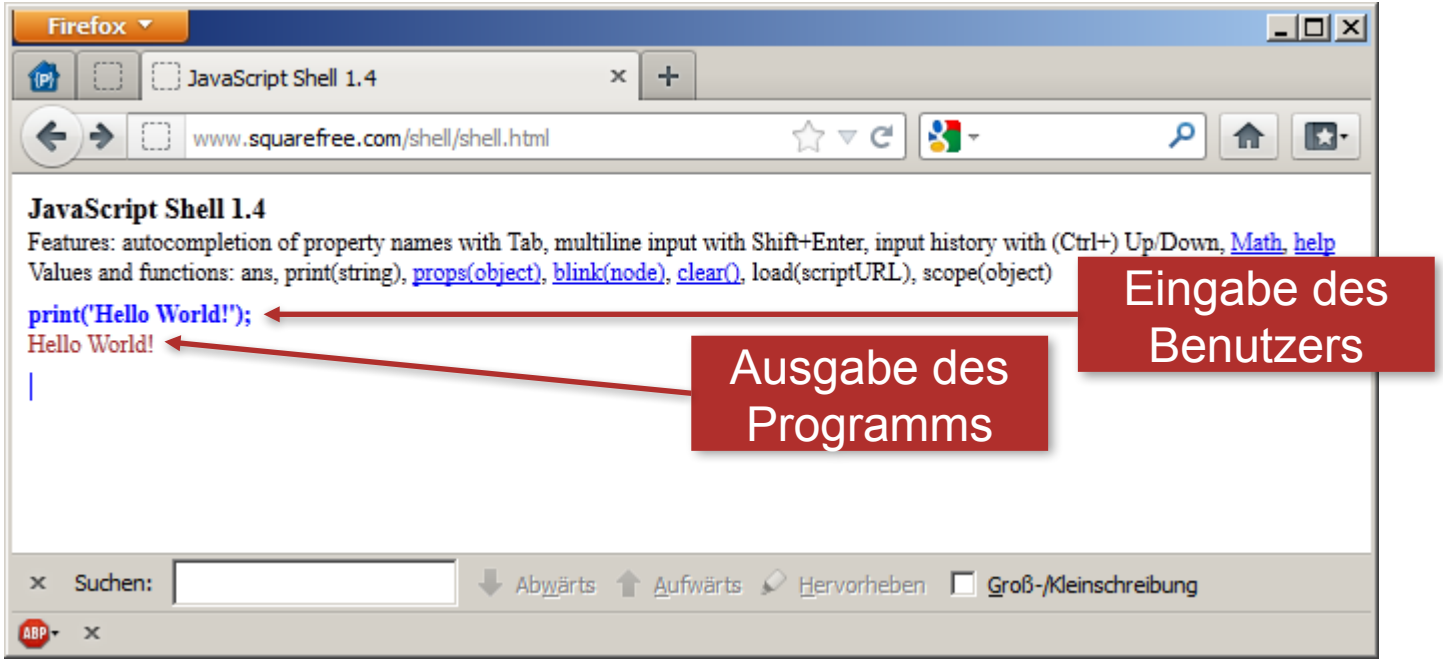


Ausgabe des SmallBASIC-Interpreters bei Ausführung des Programms



Als Textdatei gespeichertes Quellprogramm

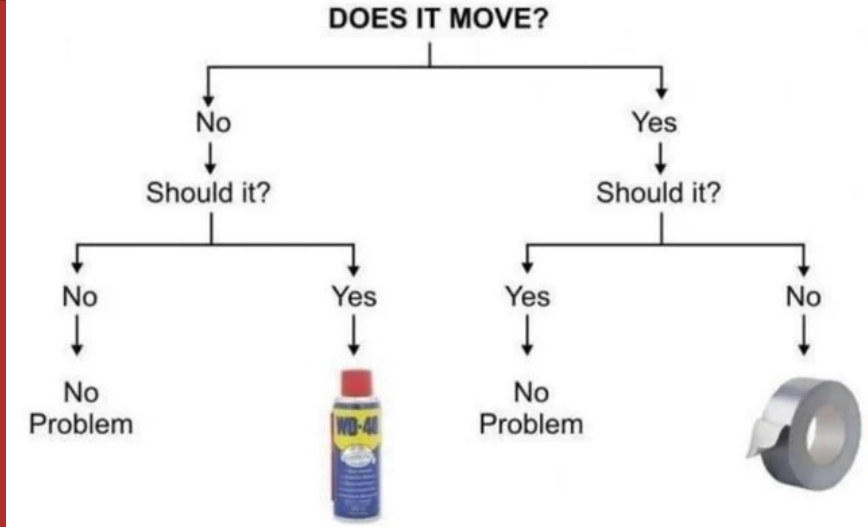
BEISPIEL FÜR DIE AUSFÜHRUNG EINES PROGRAMMS IN EINEM WEBBROWSER



PROGRAMMIERPARADIGMA

- Ein Programmierparadigma bestimmt die Ausdruckskraft einer Programmiersprache
- Ein allgemeines und grundlegendes Beispiel ist das Imperativparadigma
  - Ein Quellprogramm enthält Befehle, die eine schrittweise Lösung beschreiben
- Das prozedurale Paradigma ist eine Spezialisierung des imperativen Paradigmas.
  - Gemeinsame Schritte der Lösung werden in Unterprogrammen (sogenannten Prozeduren und Funktionen) zusammengefasst, während Unterschiede in deren Parameter umgewandelt werden.
- Abstrakte Datentypen und das objektorientierte Paradigma sind Spezialisierungen des prozeduralen Paradigmas
  - Daten und die darauf angewendeten Verfahren werden in einem Objekt zusammengefasst, das durch eine Klasse beschrieben wird

KONTROLLSTRUKTUREN





KONTROLLSTRUKTUREN

- Alle Programmiersprachen, des imperativen Paradigma, verfügen über Kontrollstrukturen zur Gliederung und Steuerung des Programmablaufs
  - Sequenz (Liste von Anweisungen, die linear ausgeführt werden)
  - Verzweigungen (Bedingungsanweisungen, Auswahlanweisungen)
  - Schleifen (Wiederholungsanweisungen)
  - (Prozeduren und Prozeduraufrufe als optionaler Bestandteil)
- Eine Programmiersprache, die mindestens eine Auswahlanweisung und mindestens eine Wiederholungsanweisung enthält, gilt als Turing-vollständig.

SEQUENZ

- Anweisungen einer linearen Abfolge werden entsprechend ausgeführt

Basic

```
10 print "One"
20 print "Two"
30 print "Three"
```

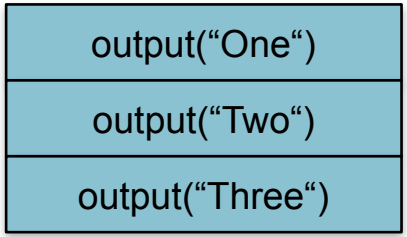
C++

```
cout << "One";
cout << "Two";
cout << "Three";
```

JavaScript

```
print('One');
print('Two');
print('Three');
```

Struktogramm



EINSEITIGE AUSWAHL

- Wenn eine Bedingung den Wert „wahr“ ergibt, wird die folgende Anweisung ausgeführt

Basic

```
if age > =18 then
print "of full age"
```

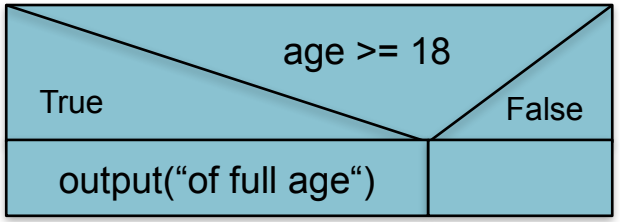
C++

```
if (age >= 18)
cout << "of full age";
```

JavaScript

```
if (age >= 18)
print('of full age');
```

Struktogramm



BEIDSEITIGE AUSWAHL

- Wenn eine Bedingung den Wert „wahr“ ergibt, wird die folgende Anweisung ausgeführt, andernfalls wird die alternative Anweisung ausgeführt.

Basic

```
if alter >= 18 then
print "of full age"
else print "minor"
```

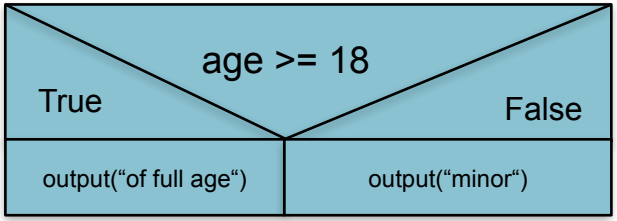
C++

```
if (age >= 18)
   cout << "of full age";
else cout << "minor";
```

JavaScript

```
if (age >= 18)
   print('of full age');
else
   print('minor');
```

Struktogramm



MEHRFACHAUSWAHL

- Wenn ein Ausdruck einen bestimmten Wert ergibt, wird die entsprechende Anweisung ausgeführt.

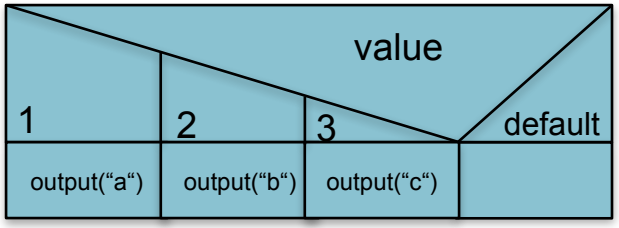
Basic C++

```
---     switch (value)
        {
          case 1: cout << "a"; break;
          case 2: cout << "b"; break;
          case 3: cout << "c"; break;
        }
```

JavaScript

```
switch (value)
{
  case 1: print('a'); break;
  case 2: print('b'); break;
  case 3: print('c'); break;
};
```

Struktogramm



KOPFGESTEUERTE SCHLEIFE – WHILE LOOP

- Solange die Bedingung den Wert „wahr“ ergibt, wird die folgende Anweisung ausgeführt
  - Wenn die Bedingung zunächst den Wert „false“ ergibt, wird die Anweisung niemals ausgeführt (bekannt als„abweisende Schleife“).

Basic

```
while counter < 10
  print counter
  counter = counter + 1
wend
```

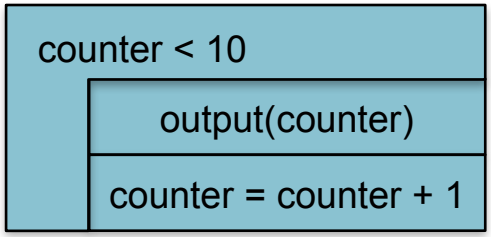
C++

```
while (counter < 10) {
   cout << counter << endl;
   counter = counter + 1;
}
```

JavaScript

```
while (counter < 10)
{
   print(counter);
   counter = counter + 1;
};
```

Struktogramm



FUßGESTEUERTE SCHLEIFE - REPEAT-/DO-LOOP

- Die Anweisung der repeat-/do-Schleife wird so lange ausgeführt, bis ihre Bedingung den Wert „wahr“ annimmt.
  - Die Anweisung der repeat-/do-Schleife mindestens einmal

Basic

```
repeat
   print counter
   counter = counter + 1
until counter = 10
```

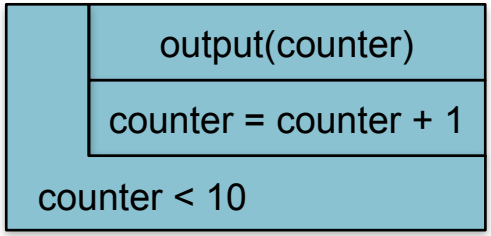
C++

```
do
{
   cout << counter << endl;
   counter = counter + 1;
} while (counter < 10);
```

JavaScript

```
do
{
   print(counter);
   counter = counter + 1;
} while (counter < 10);
```

Struktogramm



ZÄHLSCHLEIFE - COUNTING/FOR LOOP

- Eine Zählschleife, auch als „for-Schleife“ bezeichnet, verwendet eine Zählvariable, einen Start- und einen Endwert oder eine Bedingung.
  - Es gibt viele Varianten einer Zählschleife

Basic

```
for counter = 1 to 10
  print counter
next
```

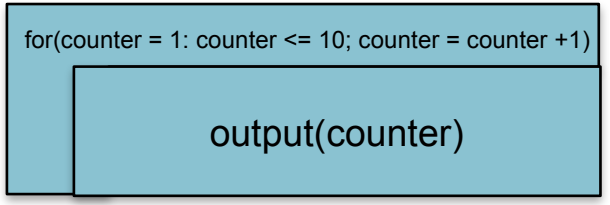
C++

```
for (counter = 1; counter <= 10; counter = counter + 1)
   cout << counter << endl;
```

JavaScript

```
for (counter = 1; counter <= 10; counter = counter +1)
   print(counter);
```

Struktogramm



PROZEDUREN UND PROZEDURAUFRUFE

- Verwandte Anweisungen oder sich wiederholende Anweisungsfolgen können zu einer Prozedur zusammengefasst werden.
- Eine Prozedur kann sogenannte Parameter haben, die bei ihrem Aufruf an die Prozedur übergeben werden.
  - In der Regel werden die Werte dieser Parameter innerhalb der Prozedur verwendet
- Wenn eine Prozedur einen Wert zurückgibt, wird sie auch als Funktion bezeichnet.
  - „Prozedur“ und „Funktion“ werden manchmal als Synonyme verwendet
  - Manchmal wird „Operation“ als Oberbegriff für Prozedur und Funktion verwendet.
- Normalerweise wird eine Prozedur aufgerufen, indem ihr Name und die erforderlichen Parameter in einer Anweisung angegeben werden.

ERSTELLUNG UND ANWENDUNG VON VERFAHREN

```
#include <iostream>
using namespace std;

int main()
{
  unsigned int z;
  cout << “natural number: ";
  cin >> z;
  unsigned int f = 1;
  for (unsigned int i = 2;i <= z;i = i + 1)
    f = f * i;
  cout << “factorial = ";
  cout << f << endl;
 }
```

Prozedur, die einen Wert zurückgibt, auch als Funktion bezeichnet

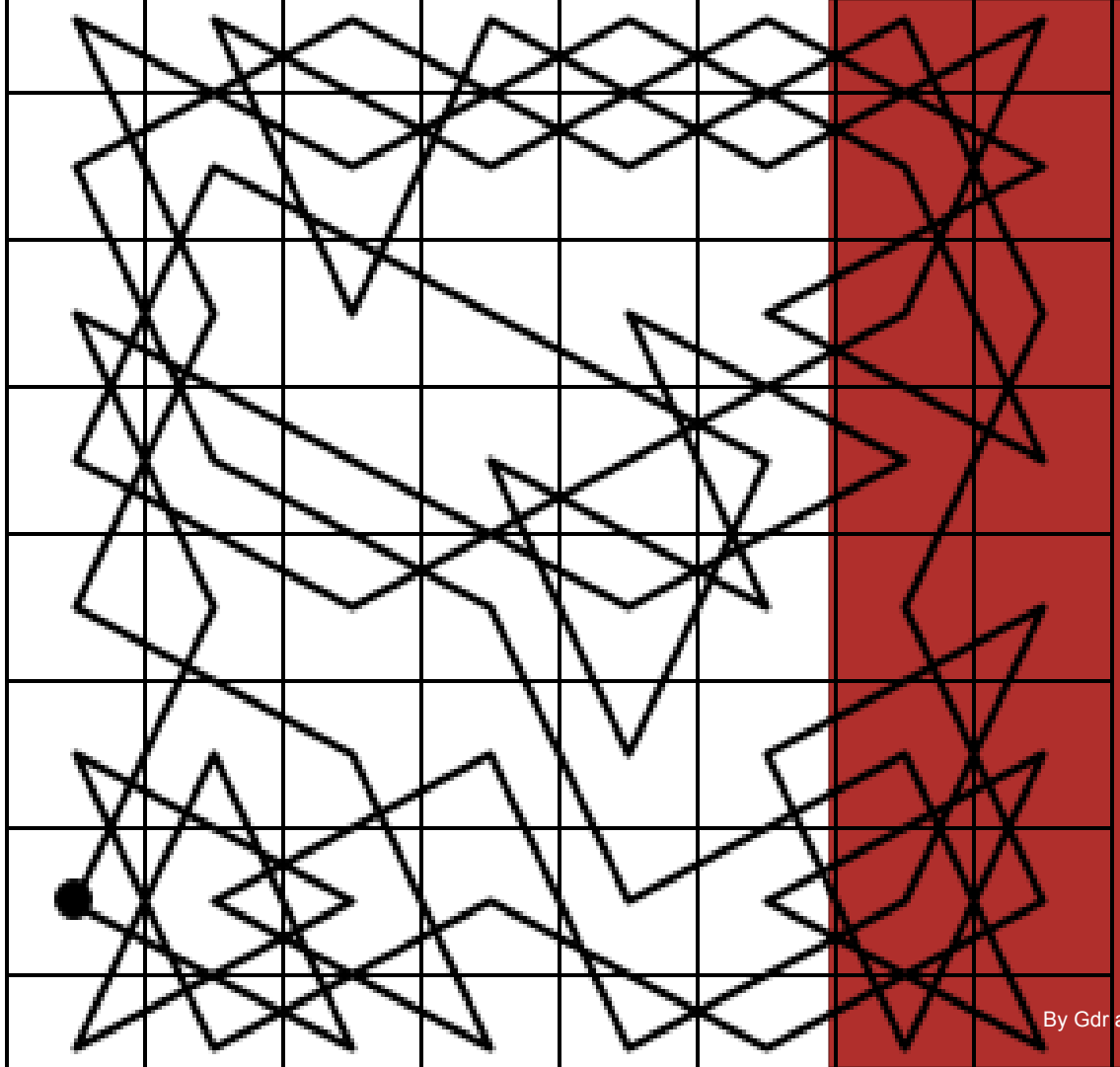
Parameter

```
#include <iostream>
using namespace std;

unsigned int factorial(unsigned int n)
{
  unsigned int f = 1;
  for (unsigned int i = 2; i <= n; i = i + 1)
    f = f * i;
  return f;
}

int main()
{
  unsigned int z;
  cout << “natural number: ";
  cin >> z;
  cout << “factorial = ";
  cout << factorial(z) << endl;
 }
```

C++



ALGORITHMEN UND HEURISTIKEN

By Gdr at English Wikipedia, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=23810552

ALGORITHMUS

- Eindeutiges und umsetzbares Verfahren zur Lösung eines Problems oder einer Problemklasse unter Verwendung einer endlichen Anzahl gezielt angeordneter Schritte
- In der Informatik weist ein Algorithmus einige wesentliche Eigenschaften auf
  - Es lässt sich mit einem Text von begrenzter Länge eindeutig ausdrücken.
  - Jeder Schritt muss ausführbar sein
  - Jeder Ausführungsschritt benötigt möglicherweise nur begrenzten Speicherplatz.
  - Es darf nur eine begrenzte Anzahl von Schritten erfordern und muss innerhalb einer begrenzten Zeit abgeschlossen sein.
  - Weitere Einschränkungen:
    - Bei Verwendung derselben Eingabedaten sollte der Algorithmus das gleiche Ergebnis liefern.
    - Bei jedem Schritt lässt sich eindeutig festlegen, welcher Schritt als Nächstes ausgeführt werden soll.

HEURISTIK

- Viele Algorithmen benötigen bei bestimmten Daten eine lange Laufzeit.
- Wenn ein Problem nicht ausreichend verstanden und nicht ausreichend formalisiert ist, kann es sein, dass noch kein effektiver Algorithmus zu seiner Lösung bekannt ist. → Heuristiken könnten hier zum Einsatz kommen
- Eine Heuristik ist ein Schätzverfahren oder eine Faustregel, mit deren Hilfe sich ein brauchbares (zulässiges) Ergebnis entweder in kurzer Zeit oder mit einfachen Mitteln ermitteln lässt.
  - Es ist nicht garantiert, dass eine Heuristik das optimale Ergebnis findet.
  - Vor einer Beschaffung werden drei Angebote von unabhängigen Anbietern eingeholt.
    - Der Bieter mit dem günstigsten Angebot gewinnt die Ausschreibung, doch dies ist nicht unbedingt das günstigste Angebot, das es gibt.

TYPEN



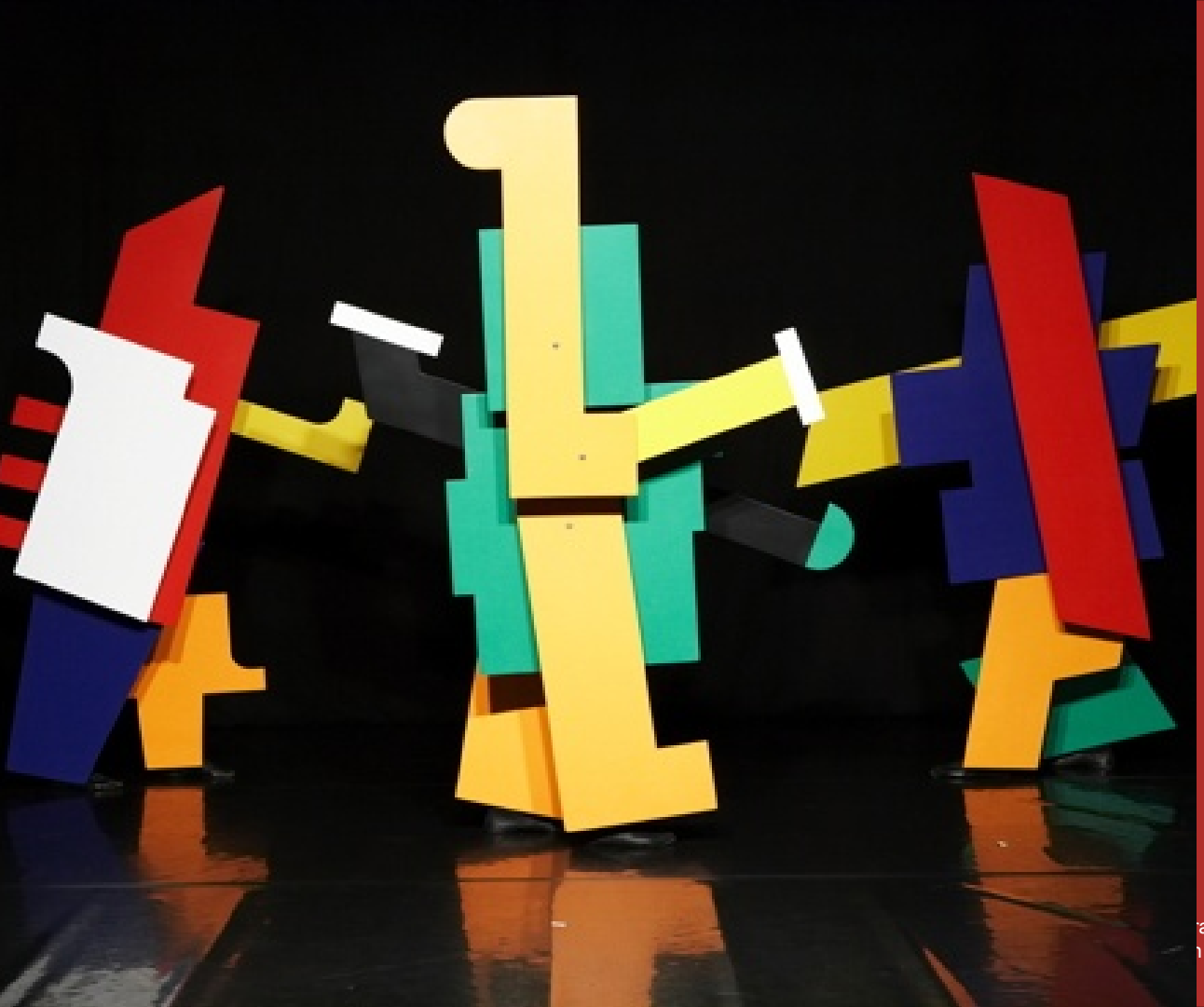
By Willi Heidelbach, CC BY 2.5, https://commons.wikimedia.org/w/index.php?curid=154912

TYPEN

- In vielen Programmiersprachen haben Daten einen bestimmten Typ
- Ein Typ definiert die Art der Daten und deren jeweiligen Wertebereich.
- Eine Variable ist ein Speicherplatz, an dem Werte gespeichert werden können.
  - Eine Variable hat einen Typ.
  - Eine Variable kann nur Werte speichern, die ihrem Typ entsprechen, z. B. hinsichtlich Größe und Wertebereich.
- Beispiele
  - int i; definiert eine Variable namens i vom Typ int; vorausgesetzt, dass int eine Größe von zwei Byte hat, kann i jede ganzzahlige Zahl im Bereich von -32.768 bis 32.767 zugewiesen werden.
  - char c; definiert eine Variable namens c vom Typ char; der Typ char hat in der Regel eine Größe von einem Byte; ihr können Zeichen zugewiesen werden, deren Werte im Bereich von 0 bis 255 oder von -128 bis 127 liegen, z. B. c = ‘x’

ELEMENTARE (INTEGRIERTE)/FUNDAMENTALE UND BENUTZERDEFINIERTE TYPEN

- Die meisten Programmiersprachen verfügen über integrierte Datentypen für Zahlen, logische Werte, Zeichen und Zeichenketten
- Diese integrierten Typen werden auch als elementare Typen bezeichnet.
- In vielen Programmiersprachen kann der Programmierer neue Datentypen definieren, sogenannte benutzerdefinierte Typen
  - Dabei kann es sich um einfache Typen handeln, wie beispielsweise Aufzählungstypen, z. B. enum Colors {red, green, blue}; oder
  - Zusammengesetzte Typen, z. B. struct Date { unsigned int day, month, year };



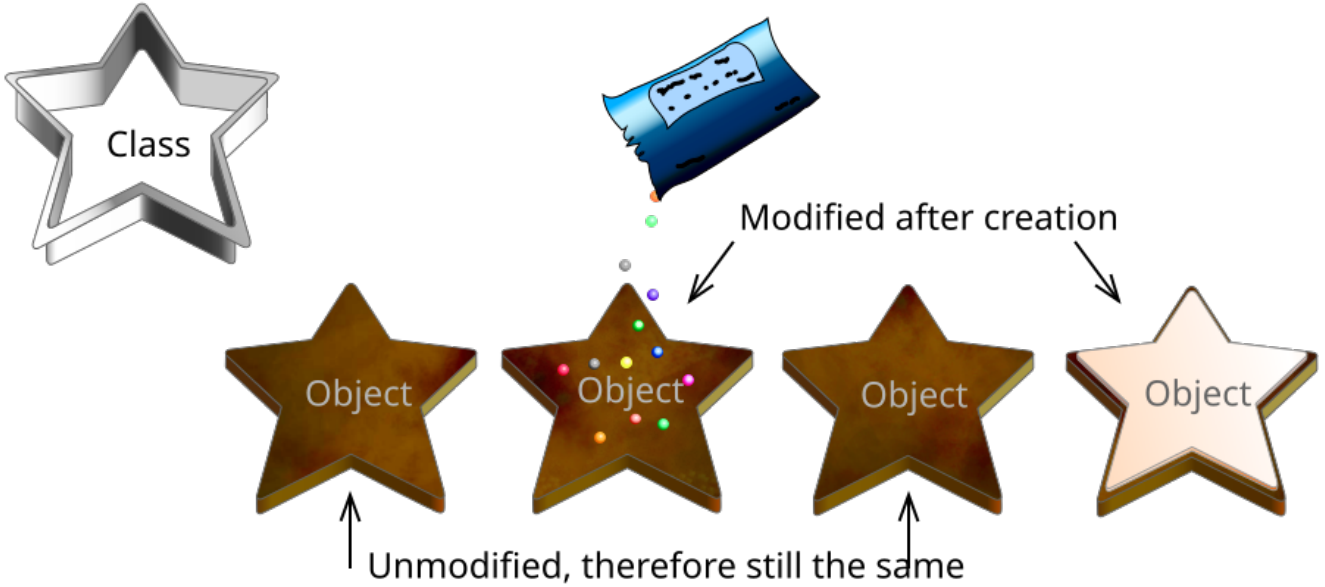
OBJEKTE

af), Theater der Klänge und Kurt Schmidt (Figurinen) - Foto einer
n Ballets durch das Düsseldorfer Theater der Klänge., CC BY 3.0,
https://commons.wikimedia.org/w/index.php?curid=78048073

ABSTRAKTE DATENTYPEN (ADT) UND KLASSEN

- Kombiniert Daten und Operationen, die an Daten bzw. Objekten durchgeführt werden können
  - Es realisiert Kapselung
  - Auf die Daten kann nur durch entsprechende Operationen zugegriffen werden.
  - Die Struktur der Daten und die Implementierungen der Operationen sind verborgen (Informationsverbergung bekannt als das „Geheimnisprinzip“)
- In der objektorientierten Programmierung erweitert die Klasse den abstrakten Datentyp.
  - Eine neue Klasse kann von einer bereits vorhandenen Klasse abgeleitet werden → dies wird als Vererbung bezeichnet
  - Eine Klasse lässt sich als eine Art Bauplan betrachten
  - Ein Exemplar einer Klasse wird auch als Instanz oder Objekt bezeichnet.

ABSTRAKTE DATENTYPEN (ADT) UND KLASSEN

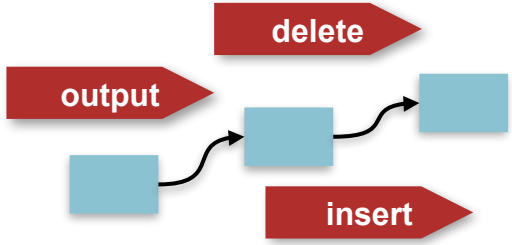


By Milesjpool, under Creative Commons, https://commons.wikimedia.org/wiki/File:CPT-Class-Object-Modification.svg

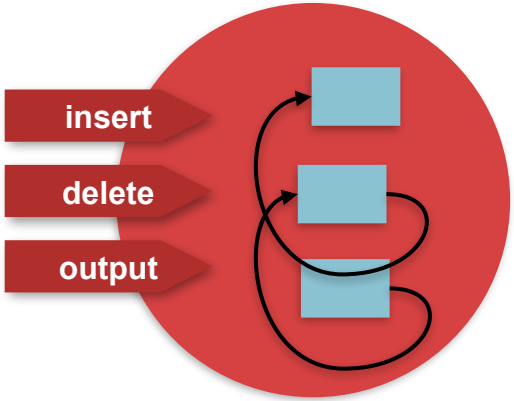
KAPSELUNG

- Eine Klasse implementiert alle Prinzipien eines abstrakten Datentyps (ADT)
- Daten und die dazugehörigen Operationen bilden eine Einheit
- *Geheimnisprinzip*
  - Die Daten sind verborgen oder zumindest nicht zugänglich
  - Auf die Daten kann nur mithilfe entsprechender Operationen zugegriffen werden

Ohne ADT oder Objektorientierung

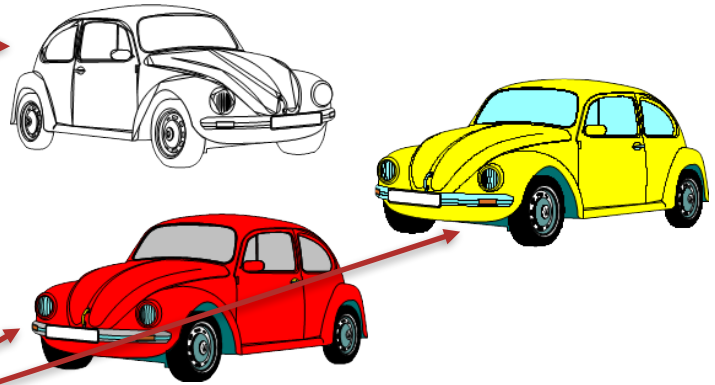


Mit ADT oder objektorientierter Programmierung



KLASSEN UND OBJEKTE

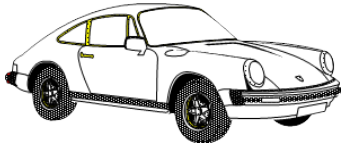
- Klassen sind Entwürfe
  - Sie legen fest, welche Daten (sogenannte Attribute) ein Objekt aufweist
  - Und seine Operationen (sogenannte Methoden)
- Ein Objekt existiert zur Laufzeit; es wird auch als Exemplar oder Instanz bezeichnet.
  - Jedes Exemplar verfügt über eigene Daten
  - Die Methoden sind allen Instanzobjekten einer Klasse gemeinsam



Attribute
Color
Engine
Methoden
start
drive

OBJEKTKOMMUNIKATION

- Objekte kommunizieren, indem sie Nachrichten senden.
- Wenn ein Objekt eine Nachricht empfängt
  - Es prüft, ob es die Nachricht versteht,
  - Und wenn ja, führt es die entsprechende Methode aus



drive
- **pressThrottle**

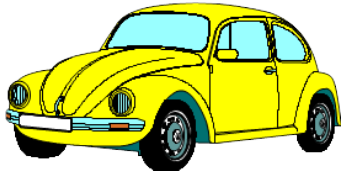
drive →





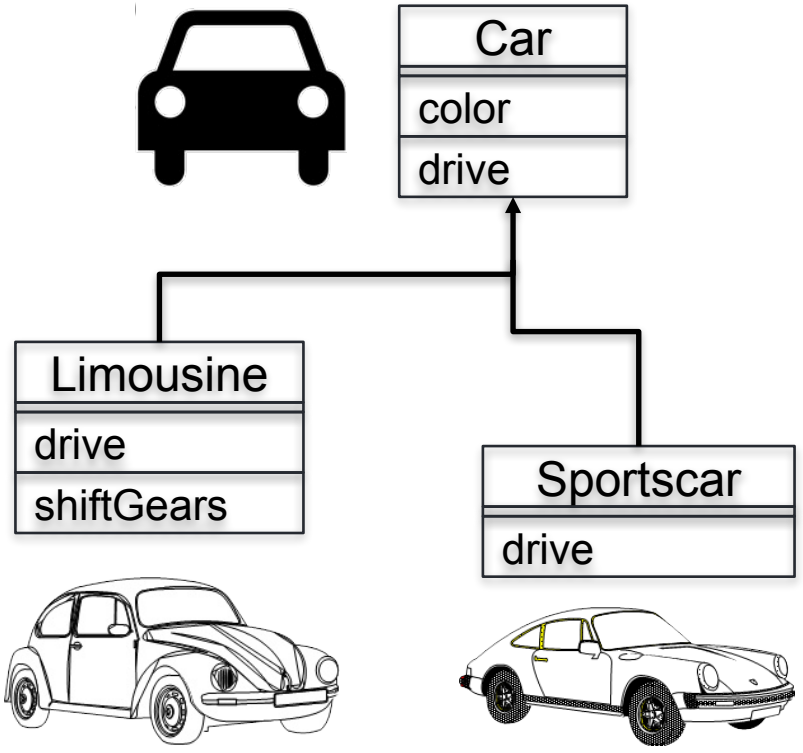
drive
- **releaseThrottle**
- **pressClutch**
- **shiftGears**
- **releaseClutch**
- **pressThrottle**

drive →



VERERBUNG

- Eine neue Klasse von einer bestehenden ableiten
- Die abgeleitete Klasse kann neue Attribute hinzufügen
- Es können neue Methoden hinzugefügt werden
- Es kann bestehende Implementierungen von Methoden ersetzen





SOFTWARE ENTWICKLUNG IN DER ZEIT VON KI

"designed by tohamina - Magnific.com"

SOFTWAREENTWICKLUNG IM WANDEL

- Klassische Softwareentwicklung: Menschen analysieren, entwerfen, implementieren und testen.
- Heute: KI unterstützt viele dieser Tätigkeiten als Werkzeug.
  - Zunehmend werden KI-Agenten auch als autonomes Teammitglied eingesetzt.
- Beispiele:
  - Code vervollständigen
  - Anforderungen zusammenfassen
  - Testfälle erzeugen
  - Fehler erklären
  - Dokumentation erstellen
- Die Verantwortung für das System bleibt beim Entwicklungsteam.

WO KOMMT KI IM SOFTWARELEBENSZYKLUS KONKRET ZUM EINSATZ?

- Anforderungen
  - User Stories formulieren
  - Anforderungen strukturieren
  - Widersprüche und fehlende Informationen erkennen
- Architektur und Design
  - Architekturvarianten vergleichen
  - Schnittstellen und Datenmodelle vorschlagen
- Implementierung
  - Boilerplate-Code erzeugen
  - APIs erklären
  - Code umstrukturieren
- Testing
  - Unit- und Integrationstests generieren
  - Testdaten erzeugen
  - Fehlerursachen analysieren
- Betrieb und Wartung
  - Logs zusammenfassen
  - Anomalien erkennen
  - Dokumentation aktualisieren

DER NEUE ENTWICKLUNGSZYKLUS: DISKUTIEREN, GENERIEREN UND VERIFIZIEREN

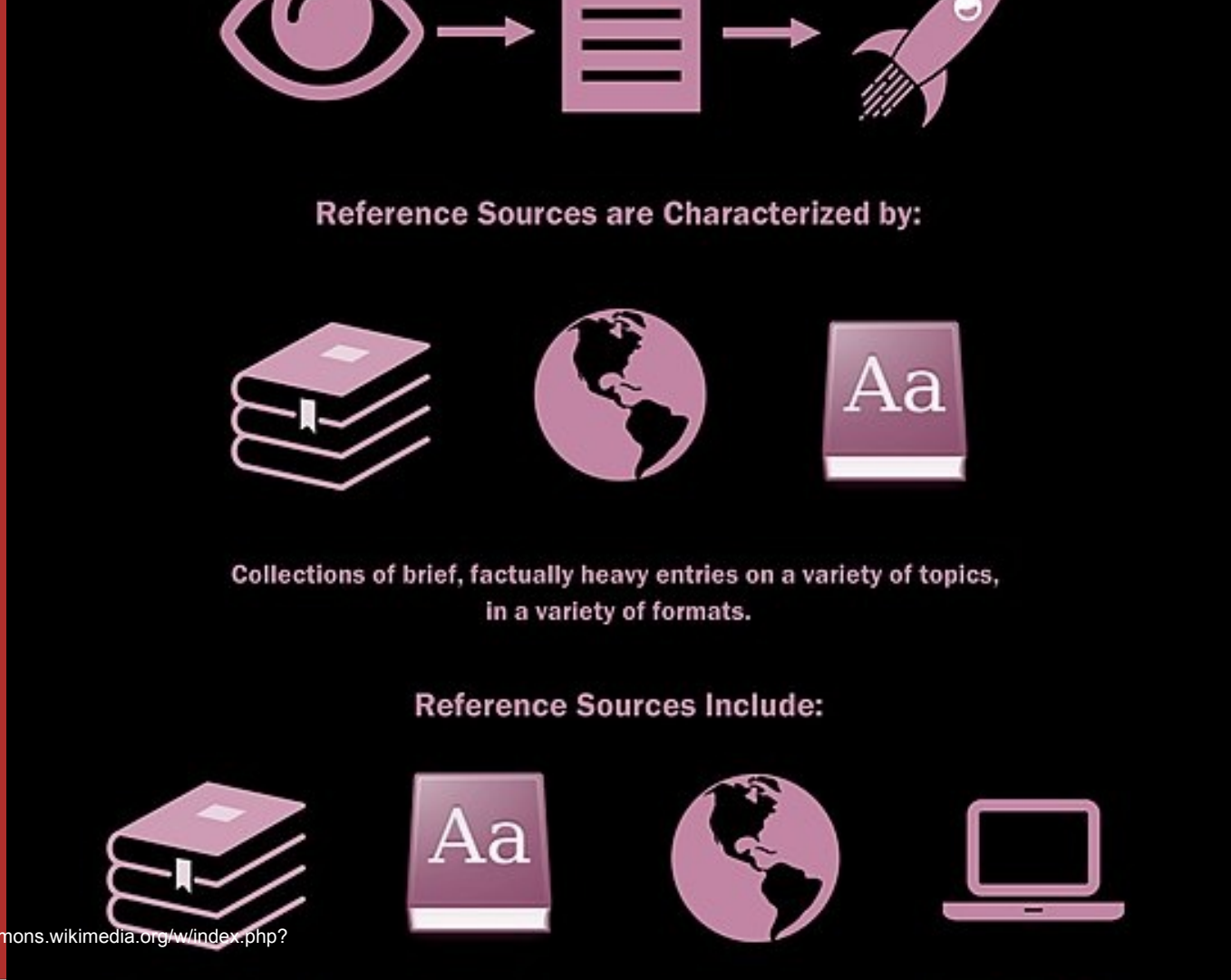
- Anforderungen und potentielle Umsetzungen werden gemeinsam mit den Agenten diskutiert
- Im Anschluss wird Code generiert
- Jeder Vorschlag sollte im Optimalfall: verstanden, kompiliert, getestet, sicherheitsbezogen geprüft, und in den Systemkontext eingeordnet werden.
- Tests können auch generiert werden um spezifische Eigenschaften zu testen
- ABER: Ein bestandener Test beweist nicht, dass die Implementierung die gewünschte Funktionalität erfüllt.
- Code Review bleibt notwendig.

CODE REVIEW BLEIBT NOTWENDIG. ABER WARUM?

- KI-generierter Code kann:
  - Syntaktisch korrekt, aber semantisch falsch sein indem Anforderungen falsch interpretiert werden
  - Randfälle oder Sicherheitslücken übersehen
  - Unsichere Bibliotheken oder APIs verwenden
  - Veraltete Informationen enthalten
  - Bestehende Architekturregeln verletzen
  - Vorurteile oder problematische Muster aus Trainingsdaten enthalten, sodass Code und Tests umfangreich erscheinen aber schwach und schwer wartbar sind

FRAGEN?

LITERATUR



By Shonnmharen - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=80669266

REFERENCES

- Balzert, Helmut, and Heide Balzert. *Software-Entwicklung*. Lehrbuch der Software-Technik. Heidelberg: Spektrum Akad. Verl, 1996.
- ———. *Software-Management, Software-Qualitätssicherung, Unternehmensmodellierung*. Lehrbuch der Software-Technik. Heidelberg: Spektrum Akad. Verl, 1998.
- Deutsch, David. *The Fabric of Reality*. London; New York: Penguin Books, 1998.
- Popper, Karl R. *Auf Der Suche Nach Einer Besseren Welt: Vorträge Und Aufsätze Aus Dreissig Jahren*. München: Piper, 1984.



UNIVERSITÄT
LEIPZIG

Dr. Nico Willert Stark
willert@wifa.uni-leipzig.de

Prof. Dr. Ulrich Eisenecker
Professur für Wirtschaftsinformatik, insbes. Softwareentwicklung
Für Wirtschaft und Verwaltung
Grimmaische Straße 12, 04109 Leipzig, Germany
T +49 341 97-33720 F +49 341 97-33729

eisenecker@wifa.uni-leipzig.de
Professur Softwareentwicklung für Wirtschaft und Verwaltung